

Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets

Formulas in depth, lookups, pivot tables, dynamic arrays, Power Query, DAX, and a championship-style case.

Approved chapter v2

Contents

Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets

Why this matters

In plain English

11.1 From a tracker to a system

11.2 Conditional aggregation in depth: the IFS family and SUMPRODUCT

11.3 The everyday function toolkit

11.4 Lookups in depth

11.5 Pivot tables

11.6 Dynamic arrays: formulas that return whole tables

11.7 Power Query: record the cleanup once, refresh forever

11.8 Power Pivot and a first taste of DAX

11.9 What-if analysis

11.10 Google Sheets power features

11.11 What Excel does that Sheets can't (and the other way round)

11.12 Spreadsheet hygiene and auditing

11.13 When to leave the spreadsheet

11.14 Competition-style Excel: a guided championship case

Common mistakes and how to spot them

In the real world: the month-end pack that was almost right

Tools

The project: rebuild a messy monthly workbook so next month is one refresh

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets

Part II – The Analyst

Chapter at a glance

You will learn to: answer business questions with the conditional functions (`SUMIF(S)` , `COUNTIF(S)` , `AVERAGEIF(S)` , `MAXIFS` , `MINIFS`) and `SUMPRODUCT` , including OR logic, date ranges, wildcards, weighted averages, and distinct counts · use the everyday toolkit of logical, text, date, and number functions · look up values in every way analysts need (exact, approximate, last match, several columns, two-way) with `XLOOKUP` and `INDEX / MATCH` · summarize thousands of rows in seconds with pivot tables, slicers, and pivot charts, and read results out of them with `GETPIVOTDATA` · write formulas that return whole lists with dynamic arrays (`FILTER` , `UNIQUE` , `SORT` , `SEQUENCE` , `LET` , `TAKE` , `VSTACK` , `LAMBDA` , `SCAN`) · combine a folder of monthly files and clean, group, merge, and reshape them with Power Query (including custom columns in M and handling broken files), so next month is one refresh · build a data model with relationships and write your first DAX measures in Power Pivot · answer "what would it take?" with Goal Seek and data tables · use Google Sheets' power features: `QUERY` , `ARRAYFORMULA` , `IMPORTRANGE` , and Connected Sheets · know what each app can't do and what to use instead · audit a workbook and recognize when a job has outgrown the spreadsheet · solve a championship-style Excel case against the clock.

Before you start: Chapter 10. Chapter 12 helps with the SQL comparisons but isn't required.

Time needed: 25–30 hours of reading and practice, spread over three to four weeks. Sections 11.2–11.4 and 11.14 reward slow, hands-on work.

Tools: Microsoft Excel for Windows (Microsoft 365) for Power Query and Power Pivot; Google Sheets for sections 11.2–11.6, 11.9, and 11.10. Where a feature exists only in one app, or only on Windows, the section says so and gives an alternative.

Practice data: `companion/ch11/` : `ch11_practice.xlsx` (Riverstone's 330 order lines for 2025 as a clean Sales table, with Customers, Products, Targets, TargetsWide, and RebateBands sheets), the folder `monthly_exports/` (twelve monthly CSV files), `month_end_pack_2025_messy.xlsx` (the workbook from this chapter's story), `pq_practice/` (a broken export for section 11.7), and `challenge/` (the two competition-style cases in section 11.14). Every total matches Chapter 10's tracker and Chapter 13's database: net revenue excluding cancelled orders is ₹4,335,471.

Why this matters

In Chapter 10 you built a monthly tracker by hand. It works, and it's correct. Now imagine next month. A new export arrives. Someone has to import it, paste it under last month's rows, extend every formula, check that the pivot picked up the new rows, and fix the one chart whose range stopped at December. Multiply that by every report a team produces, and you understand why so many analysts spend the first week of each month copying and pasting.

This chapter is about removing that week. The tools here (lookups that don't break, pivot tables, formulas that return whole tables, Power Query, and data models) let you build a workbook **once** and **refresh** it forever after. They're also the tools interviewers test when a job description says "advanced Excel": you'll be asked to build a pivot table live, explain `INDEX / MATCH`, or describe how you'd automate a monthly report.

There's a second, quieter reason. Workbooks built by copy and paste accumulate errors nobody sees: a range that stops a few rows short, a number typed in from an email, a formula that includes cancelled orders in some months but not others. This chapter's story is about exactly such a workbook, and the auditing habits in section 11.12 are how you find those errors before your manager does.

Finally, you'll learn where the spreadsheet stops. Knowing when to move a job to SQL, Python, or Power BI is as much a mark of skill as knowing every Excel function.

In plain English

Picture a school office preparing the **annual results**.

Lookups are the office assistant with the class register: given a roll number, she finds the student's name, their section, or, for fee concessions, which **band** their family income falls into. Chapter 10 taught her to find exact matches; now she learns to find the right band, the most recent entry, and a value at the crossing of a row and a column.

A **pivot table** is the board on the principal's wall where you slide cards around: put "class" cards down the side, "subject" cards across the top, and every square shows the average mark. Rearrange the cards and the board redraws itself.

Dynamic arrays are a photocopier that prints a whole list from one instruction: "every student in section B, sorted by marks". Change a mark and the printout updates.

Power Query is a recorded routine for the clerk who collects mark sheets from twelve teachers every term: open each file, fix the date format, remove absentees, stack them into one list. Record it once; next term, drop the new files in the tray and press one button.

A **data model** is a set of linked registers (students, subjects, calendar) instead of one enormous sheet, and **DAX measures** are formulas like "pass percentage" written once and reused on any board.

Goal Seek answers the reverse question: "what mark does this student need in the final exam to pass overall?"

Here's the mapping:

- The assistant's lookups are `XLOOKUP` and `INDEX / MATCH`; the bands are an **approximate match**.
 - The sliding-card board is a **pivot table**: **Rows**, **Columns**, **Values**, and **Filters** areas.
 - The photocopier is a **dynamic array** formula that **spills** its results into neighboring cells.
 - The recorded routine is a **query** in Power Query, made of **applied steps**.
 - The linked registers are a **data model** with **relationships**; "pass percentage" is a **measure**.
 - The reverse question is **what-if analysis**.
-

11.1 From a tracker to a system

Open `month_end_pack_2025_messy.xlsx`. It's the kind of workbook you'll inherit in your first job: one sheet per month, each pasted from that month's ERP export, and a Summary sheet that pulls each month's total with its own formula. The Summary says Riverstone reached **100.7%** of its 2025 target.

Chapter 10's tracker, built from the same data, says **102.3%**. One of them is wrong, and you'll find out which (and why) in this chapter's story. For now, notice what the structure itself forces on whoever maintains it:

Manual monthly workbook	Refreshable workbook
One sheet per month; data pasted in	One table with every month; data loaded by a query
A different formula per month on the Summary	One pivot table or one formula for all months
Ranges that must be extended by hand	Tables and queries that grow with the data
Rules (exclude cancelled) repeated in every formula	Rules applied once, in one place
Checking means re-adding by hand	Check cells that compare totals automatically
Next month: an afternoon	Next month: Data → Refresh All

Three design ideas run through the rest of the chapter:

1. **Keep data long, not wide.** One row per order line, with the month as a *column*, not a *sheet*. Every tool in this chapter (pivot tables, `FILTER`, `QUERY`, Power Query, DAX) expects data in this shape. It's the same idea as Chapter 12's tables.
2. **Separate data, calculations, and outputs.** Raw data on one sheet or query, lookups and rules in another, and the report on its own sheet. Section 11.12 turns this into a checklist.
3. **Make the rules explicit and single.** "Net revenue excludes cancelled orders" should live in exactly one place.

Open `ch11_practice.xlsx`. Its **Sales** sheet already follows these ideas: 330 order lines, 19 columns, one row per line, with the lookups from Chapter 10 filled in as values.

Column	Field	Column	Field
A	<code>order_id</code>	K	<code>month_start</code>
B	<code>order_date</code>	L	<code>quarter (Q1-Q4)</code>
C	<code>customer_code</code>	M	<code>customer_name</code>
D	<code>product_id</code>	N	<code>segment</code>
E	<code>quantity</code>	O	<code>city</code>
F	<code>unit_price</code>	P	<code>product_name</code>
G	<code>discount_pct</code>	Q	<code>category</code>
H	<code>status</code>	R	<code>unit_cost</code>
I	<code>sales_rep</code>	S	<code>gross_margin</code>
J	<code>net_revenue</code>		

`gross_margin` is net revenue minus product cost (quantity × unit cost), the same rule as Chapter 12. Before you start, click inside the data and convert it to a table named **Sales** (**Excel: Insert → Table**, then **Table Design → Table Name**; **Sheets: Format → Convert to table**, then rename it). Most formulas in this chapter refer to it as `Sales[column]`.

11.2 Conditional aggregation in depth: the IFS family and SUMPRODUCT

Chapter 10 introduced `SUMIFS`, `COUNTIFS`, and `AVERAGEIFS`. In real workbooks these functions do more work than any others: most finance and sales reports are grids of them. This section takes them to full depth, adds `SUMPRODUCT` for the questions they can't answer, and ends with the patterns (ranks, running totals, grids, distinct counts) you'll build again and again.

The formulas in this section use plain ranges on the **Sales** sheet (`Sales!J2:J331` and so on), so they work in every version of Excel and in Google Sheets. With the data converted to a table, `Sales[net_revenue]` does the same job and grows with the data. As a reminder: **H** is **status**, **J** is **net_revenue**, **N** is **segment**.

One pair of arguments per condition; all conditions must hold (AND). Result: 1,488,773.75

`=SUMIFS(` `Sales!J2:J331` `,` `Sales!N2:N331` `,` `"Retail"` `,` `Sales!H2:H331` `,` `"<>Cancelled"` `)`

sum range (always first)
criteria range 1
criterion 1
criteria range 2
criterion 2

criterion	meaning	Riverstone result
"Retail"	equals (not case-sensitive)	118 lines
"<>Cancelled"	not equal	326 lines
">=50000"	compare a number (no commas)	3 lines
">=&G1"	operator in quotes, cell outside	depends on G1
">=&DATE(2025,7,1)"	dates: build them with DATE	Q3 ₹1,120,289 (with <= 30 Sep)
"*Box*"	wildcards: * any text, ? one character	169 lines
" " / "<"	blank / not blank	19 / 311 rep cells

Figure 11.1 – How `SUMIFS` reads its arguments. Every criteria range must be the same size as the sum range; every criterion is text, even when it compares numbers or dates.

The family at a glance

Function	Arguments	Returns
<code>SUMIF</code>	(range, criterion, [sum_range])	Sum where one condition holds
<code>SUMIFS</code>	(sum_range, criteria_range1, criterion1, ...)	Sum where all conditions hold
<code>COUNTIF</code>	(range, criterion)	Count of cells meeting one condition
<code>COUNTIFS</code>	(criteria_range1, criterion1, ...)	Count of rows meeting all conditions
<code>AVERAGEIF</code>	(range, criterion, [average_range])	Average where one condition holds
<code>AVERAGEIFS</code>	(average_range, criteria_range1, criterion1, ...)	Average where all hold
<code>MAXIFS</code> , <code>MINIFS</code>	(max_range, criteria_range1, criterion1, ...)	Largest or smallest where all hold (Excel 2019 or later, Sheets)

Watch out: `SUMIF` and `SUMIFS` put the sum range in different places. `SUMIF` takes it **last**; `SUMIFS` takes it **first**. Swapping them is the most common error in this family. Many analysts use only the `S` versions, even for one condition, so the argument order never changes.

```
=SUMIF(Sales!N2:N331,"Retail",Sales!J2:J331) → 1551423.75
=SUMIFS(Sales!J2:J331,Sales!N2:N331,"Retail",Sales!H2:H331,"<>Cancelled") → 1488773.75
```

The first formula has no status condition, so it includes both cancelled orders (both were Retail). The second is the correct Retail revenue. Adding one pair of arguments changed the answer by ₹62,650.

The criteria cookbook

A criterion is always **text** that describes a test. Once you know the patterns, you can write almost any condition.

You want	Criterion	Riverstone example	Result
Equal to text	"Retail"	Retail lines (all statuses)	COUNTIF(N, "Retail") → 118
Not equal to	"<>Cancelled"	Valid lines	326
Compare a number	">=50000"	Lines worth ₹50,000 or more	3
Compare with a cell	">=&G1"	Lines above the threshold in G1	depends on G1
Between two values	two pairs on the same range	Lines from ₹20,000 to under ₹50,000 (valid)	59
Between two dates	">=&DATE(2025,7,1) and "<=&DATE(2025,9,30)"	Q3 valid revenue	1,120,289
Contains text	"*Box*"	Lines whose product name contains "Box"	169
Pattern with single characters	"Storage Box ??L"	Storage Box 10L and 25L lines	125
A literal * or ?	"~*"	Cells containing an asterisk	—
Blank	" "	Lines with no sales rep	19
Not blank	"<>"	Lines with a sales rep	311

The full formulas for the less obvious ones:

```
=COUNTIFS(Sales!J2:J331,">=20000",Sales!J2:J331,"<50000",Sales!H2:H331,"<>Cancelled") → 59
=SUMIFS(Sales!J2:J331,Sales!B2:B331,">=&DATE(2025,7,1),Sales!B2:B331,"<=&DATE(2025,9,30),Sales!H2:H331,"<>Cancelled") → 1120289
=COUNTIF(Sales!P2:P331,"*Box*") → 169
=COUNTIF(Sales!P2:P331,"Storage Box ??L") → 125
=COUNTIFS(Sales!I2:I331,"<>") → 311
```

Rules that save hours:

- **Operators go inside the quotes; cell references go outside.** ">=&G1" joins the operator to the cell's value. ">=G1" compares with the text "G1" and returns 0.
- **Criteria aren't case-sensitive.** "retail" matches Retail. For case-sensitive tests, use SUMPRODUCT with EXACT (below).
- **Don't type thousands separators in criteria.** ">=50,000" is read as text, not as a number. Write ">=50000".
- **Dates in criteria need DATE or a cell.** ">=01-07-2025" depends on the computer's regional settings; ">=&DATE(2025,7,1)" doesn't.

- **Numbers stored as text don't match numbers.** A criterion of 101 doesn't count a cell holding the text "101" in every situation. Fix the types first (Chapter 10, section 10.3).

OR conditions

SUMIFS combines its conditions with **AND**. For **OR** on the **same** column, give it a list of criteria in curly brackets (an **array constant**) and add up the results:

```
=SUM(SUMIFS(Sales!J2:J331,Sales!N2:N331,{"Retail","Hospitality"},Sales!H2:H331,"<>Cancelled")) → 2632812.5
```

SUMIFS returns two numbers, one for Retail (₹1,488,773.75) and one for Hospitality (₹1,144,038.75), and SUM adds them. In Google Sheets, wrap the formula in ARRAYFORMULA or use two SUMIFS added together.

For OR across **different** columns, adding two COUNTIFS double-counts the rows that meet both conditions. "How many valid lines are Wholesale OR have a quantity of 50 or more?"

```
=COUNTIFS(Sales!N2:N331,"Wholesale",Sales!H2:H331,"<>Cancelled")+COUNTIFS(Sales!E2:E331,">=50",Sales!H2:H331,"<>Cancelled") → 151 (wrong)
```

105 Wholesale lines plus 46 large-quantity lines is 151, but 6 lines are both. The right answer needs SUMPRODUCT.

SUMPRODUCT: conditions the IFS family can't express

SUMPRODUCT multiplies arrays item by item and adds the results. A test such as (Sales!N2:N331="Wholesale") produces an array of TRUE and FALSE values, and multiplying turns them into 1s and 0s. That makes SUMPRODUCT the universal conditional tool, and it works in every version of Excel and in Sheets.

```
=SUMPRODUCT(((Sales!N2:N331="Wholesale")+(Sales!E2:E331>=50))>0)*(Sales!H2:H331<>"Cancelled")) → 145
```

How it works:

- (A)+(B) is 1 when one test is true and 2 when both are. >0 turns both cases into TRUE, which is **OR** without double counting.
- *(Sales!H2:H331<>"Cancelled") is **AND** with the status rule.
- SUMPRODUCT adds the resulting 1s: **145** lines. Replace the last part with *(Sales!H2:H331<>"Cancelled")*Sales!J2:J331 and it adds their revenue instead: ₹2,533,048.50.

Other questions only SUMPRODUCT (or FILTER, section 11.6) can answer:

A condition on a calculation, such as the month of a date, without a helper column:

```
=SUMPRODUCT((MONTH(Sales!B2:B331)=11)*(Sales!H2:H331<>"Cancelled")*Sales!J2:J331) → 633408
```

SUMIFS can't apply MONTH to its criteria range; SUMPRODUCT can.

A weighted average. "What was Riverstone's average discount?" The plain average of the discount column treats a ₹546 line and a ₹56,700 line equally:

```
=AVERAGEIFS(Sales!G2:G331,Sales!H2:H331,"<>Cancelled") → 4.06
```

Weighting each line's discount by its value before discount gives the discount the business actually gave:

```
=SUMPRODUCT((Sales!H2:H331<>"Cancelled")*Sales!E2:E331*Sales!F2:F331*Sales!G2:G331)
/SUMPRODUCT((Sales!H2:H331<>"Cancelled")*Sales!E2:E331*Sales!F2:F331) → 4.69
```

The weighted average discount is **4.69%**, not 4.06%, because the big lines carry the bigger discounts. When someone asks for "the average", ask what it should be weighted by.

A distinct count. "How many orders (not lines) were valid?"

```
=SUMPRODUCT((Sales!H2:H331<>"Cancelled")/COUNTIFS(Sales!A2:A331,Sales!A2:A331)) → 173
```

COUNTIFS(A, A) returns, for every row, how many rows share its order ID. An order with three lines gets 3 on each line, so each line contributes 1/3 and the order adds up to 1. The same trick with the customer column,

=SUMPRODUCT(1/COUNTIF(Sales!M2:M331,Sales!M2:M331)) , counts **23** customers. In Microsoft 365,

=COUNTA(UNIQUE(...)) is simpler (section 11.6).

Watch out: SUMPRODUCT on whole columns. SUMPRODUCT((A:A="x")*B:B) makes Excel multiply a million rows. Use exact ranges or table columns. Every array in a SUMPRODUCT must also be the same size, or it returns #VALUE! .

Averages that mean something: line, order, and customer

"Average sale" has at least three meanings, and they give very different answers:

Question	Formula idea	Result
Average line value (valid)	AVERAGEIFS(J, H, "<>Cancelled")	₹13,298.99
Average Wholesale line	AVERAGEIFS(J, N, "Wholesale", H, "<>Cancelled")	₹16,215.80
Average order value (AOV)	valid revenue ÷ distinct valid orders	₹25,060.53
Average Wholesale order	Wholesale revenue ÷ distinct Wholesale orders	₹30,957.43

The AOV formula combines the two tools:

```
=SUMIFS(Sales!J2:J331,Sales!H2:H331,"<>Cancelled")/SUMPRODUCT((Sales!H2:H331<>"Cancelled")/COUNTIFS(Sales!A2:A331,Sales!A2:A331)) → 25060.53
```

Chapter 23 defines average order value as a KPI; the grain matters there too.

Watch out: AVERAGEIFS with no matches. If no row meets the conditions, AVERAGEIFS returns #DIV/0! . Wrap it: =IFERROR(AVERAGEIFS(...),"no sales") . AVERAGEIFS also ignores blank cells in the average range but counts zeros, so a zero typed in place of "no data" pulls averages down.

MAXIFS and MINIFS

```
=MAXIFS(Sales!J2:J331,Sales!N2:N331,"Retail",Sales!H2:H331,"<>Cancelled") → 49875
=MAXIFS(Sales!B2:B331,Sales!N2:N331,"Wholesale") → 2025-12-17
=MINIFS(Sales!G2:G331,Sales!G2:G331,">0") → 5
```

The biggest Retail line was ₹49,875; the last Wholesale order came on 17 December (format the cell as a date); the smallest discount actually given was 5%. `MAXIFS` on a date column is the cleanest way to find "latest order per customer" in a customer list.

Patterns you'll build every week

A two-dimensional grid with mixed references. Segments down column F (rows 16–18), quarters across row 15 (G to J):

```
G16: =SUMIFS(Sales!$J$2:$J$331, Sales!$N$2:$N$331, $F16, Sales!$L$2:$L$331, G$15, Sales!$H$2:$H$331, "<>Cancelled")
```

Fill it across and down. `$F16` always reads the segment column; `G$15` always reads the quarter row. Twelve cells from one formula give the same grid as Figure 11.3's pivot table, and the grid stays exactly where you put it, which is why finance teams often prefer `SUMIFS` grids to pivots for fixed-layout reports.

A running total (year to date). In a monthly table with month starts in `F2:F13`:

```
I2: =SUMIFS(Sales!$J$2:$J$331, Sales!$B$2:$B$331, "<="&EOMONTH(F2,0), Sales!$H$2:$H$331, "<>Cancelled")
```

September's row shows ₹2,581,168.75 and December's ₹4,335,471, the same as the DAX `TOTALYTD` measure in section 11.8. The alternative, `=SUM($G$2:G2)` on a monthly column, works too, and uses Chapter 10's expanding range.

A rank. With monthly revenue in `G2:G13`:

```
H2: =RANK.EQ(G2, $G$2:$G$13, 0)
```

October ranks **1** and June **12**. `RANK.EQ` gives tied values the same rank (two lines worth ₹51,520 both rank 2, and the next rank is 4). To rank *within a group*, such as customers within their segment, use `COUNTIFS`: `=COUNTIFS(segment_col, segment_cell, revenue_col, ">"&revenue_cell)+1`.

Share of total. `=G2/SUM($G$2:$G$13)`, or for a segment, `=SUMIFS(... "Wholesale"...)/SUMIFS(... all valid...)` → 0.393 (39.3%).

A total that ignores errors. If a column contains an `#N/A` from a failed lookup, `SUM` returns `#N/A`. `AGGREGATE` can skip errors: `=AGGREGATE(9, 6, range)`, where **9** means `SUM` and **6** means "ignore error values". It also ignores rows hidden by filters with the right option. Use it for display totals, and fix the errors themselves before the report goes out.

SQL link. Every formula in this section is a `WHERE` plus an aggregate. The grid is `GROUP BY segment, quarter`; the running total is `SUM(...) OVER (ORDER BY month)` in Chapter 13; the distinct count is `COUNT(DISTINCT order_id)`. When you find yourself building a 50-by-50 grid of `SUMIFS`, that's the signal that SQL or a pivot table would do it better.

11.3 The everyday function toolkit

Excel has more than 500 functions, but a working analyst uses about 60 of them constantly. Chapter 10 covered the basics. This section covers the rest of that core set, grouped by the job they do, each with a Riverstone example you can type into a spare sheet. The Google Sheets column says whether Sheets has the same function.

Decisions: IF, IFS, SWITCH, CHOOSE, IFERROR, IFNA

Function	Example	Result	Sheets
IF	=IF(J5>=30000,"Large","Not large")	Large (row 5 is ₹32,062.50)	Yes
IFS	=IFS(J2>=30000,"Large",J2>=10000,"Medium",TRUE,"Small")	Small (row 2 is ₹2,900)	Yes
SWITCH	=SWITCH(H2,"Delivered","Closed","Shipped","In transit","Pending","Open","Cancelled","Void")	Closed	Yes
CHOOSE	=CHOOSE(ROUNDUP(MONTH(B2)/3,0),"Q1","Q2","Q3","Q4")	Q1	Yes
IFERROR	=IFERROR(G2/B2,0)	0 when B2 is 0	Yes
IFNA	=IFNA(MATCH("9999",Customers!A2:A25,0),"not found")	not found	Yes
AND, OR, NOT, XOR	=NOT(H2="Cancelled")	TRUE	Yes

- **SWITCH** compares one value with a list of exact matches, which is neater than nested IF s when you're translating codes (status codes, region codes, grades). An optional last argument is the default.
- **CHOOSE** picks the *n*th item from a list; here ROUNDUP(MONTH/3) turns a month into a quarter number, and CHOOSE turns 1–4 into labels.
- **IFNA** catches only #N/A, the "not found" error, and lets other errors through. That's safer than IFERROR for lookups, because a typo in a range name still shows up as #REF! or #NAME?.

Applied to all valid lines, the IFS size bands give **158** Small (under ₹10,000), **142** Medium, and **26** Large (₹30,000 or more).

Text: SEARCH, TEXTBEFORE, TEXTAFTER, TEXTSPLIT, TEXTJOIN, REPT, EXACT

Function	Example	Result	Sheets
SEARCH	=SEARCH("box","Lunch Box Set")	7 (not case-sensitive, allows wildcards)	Yes
FIND	=FIND("box","Lunch Box Set")	#VALUE! (case-sensitive)	Yes
TEXTBEFORE	=TEXTBEFORE("Neha Kulkarni"," ")	Neha	No (use LEFT + FIND)
TEXTAFTER	=TEXTAFTER("Neha Kulkarni"," ")	Kulkarni	No (use MID + FIND)
TEXTSPLIT	=TEXTSPLIT("02.01.2025	SO10001	C0002","
TEXTJOIN	=TEXTJOIN(", ",TRUE,IF(Customers!D2:D25="Wholesale",Customers!B2:B25,""))	Coastal Foods, Northgate Distributors, ...	Yes

Function	Example	Result	Sheets
REPT	=REPT(" "	",ROUND(0.393*20,0))"	
EXACT	=EXACT("0014","0014")	TRUE (case-sensitive comparison)	Yes
CLEAN	=CLEAN(A2)	Removes non-printing characters	Yes
NUMBERVALUE	=NUMBERVALUE("4.335.471,00",".",",",".")	4335471 (European separators)	No (use SUBSTITUTE + VALUE)

TEXTBEFORE, TEXTAFTER, and TEXTSPLIT are Microsoft 365 functions; they replace most LEFT / MID / FIND combinations. The TEXTJOIN example lists every Wholesale customer in one cell: Coastal Foods, Northgate Distributors, Harbour Traders, Deccan Packaging, Western Logistics, Prime Wholesale. In older Excel it needs **Ctrl+Shift+Enter**; in Microsoft 365 it works as typed; in Google Sheets, wrap it in ARRAYFORMULA. The championship case at the end of this chapter uses these text functions to decode a whole shipment log.

Dates: EDATE, WORKDAY.INTL, NETWORKDAYS.INTL, WEEKNUM, ISOWEEKNUM, DATEDIF

Function	Example	Result	Sheets
EDATE	=EDATE(DATE(2025,1,31),1)	2025-02-28	Yes
WORKDAY.INTL	=WORKDAY.INTL(DATE(2025,12,24),3,11)	2025-12-27	Yes
	=WORKDAY.INTL(DATE(2025,12,24),3,11,DATE(2025,12,25))	2025-12-29	Yes
NETWORKDAYS.INTL	=NETWORKDAYS.INTL(DATE(2025,12,1),DATE(2025,12,31),11)	27	Yes
WEEKNUM	=WEEKNUM(DATE(2025,12,29),1)	53	Yes
ISOWEEKNUM	=ISOWEEKNUM(DATE(2025,12,29))	1	Yes
DATEDIF	=DATEDIF(DATE(2025,1,24),DATE(2025,12,5),"m")	10 (whole months)	Yes
	=DATEDIF(DATE(2025,1,24),DATE(2025,12,5),"d")	315 (days)	Yes

- **EDATE** adds whole months and clamps to the month's last day: one month after 31 January is 28 February. It's the right tool for "due one month after invoice".
- **WORKDAY.INTL** and **NETWORKDAYS.INTL** take a **weekend code**. 1 (the default) is Saturday–Sunday; 11 is **Sunday only**, common for Indian businesses that work six days a week. A delivery promised "3 working days" after 24 December, working Monday to Saturday, lands on 27 December; with 25 December as a holiday, on 29 December. December 2025 has 27 Monday-to-Saturday working days.
- **Week numbers are a trap**. WEEKNUM (type 1) counts from 1 January, so 29 December 2025 is week 53. ISOWEEKNUM follows the ISO 8601 standard, in which weeks start on Monday and week 1 contains the year's first Thursday, so the same date is week 1 of 2026. Agree on one system before building a weekly report.
- **DATEDIF** is an old, undocumented-in-menus function that still works: Sharma Hardware's first and last 2025 orders were 10 whole months (315 days) apart. Units: "y", "m", "d", "ym", "md".

Numbers and statistics: LARGE, SMALL, RANK.EQ, PERCENTILE, MODE, rounding

Function	Example (valid and cancelled lines, or)	Result
LARGE	=LARGE(Sales!J2:J331,2)	51520 (the second-largest line)

Function	Example (valid and cancelled lines, or)	Result
SMALL	=SMALL(Sales!J2:J331,1)	546.25
RANK.EQ	=RANK.EQ(51520,Sales!J2:J331,0)	2
MEDIAN	=MEDIAN(Sales!J2:J331)	10212.5
PERCENTILE.INC	=PERCENTILE.INC(Sales!J2:J331,0.9)	26600 (90% of lines are at or below this)
QUARTILE.INC	=QUARTILE.INC(Sales!J2:J331,1)	6127.5
MODE.SNGL	=MODE.SNGL(Sales!E2:E331)	15 (the most common quantity)
MROUND	=MROUND(47,10)	50 (nearest multiple)
CEILING.MATH	=CEILING.MATH(45,10)/10	5 (cartons of 10 needed for 45 units)
FLOOR.MATH	=FLOOR.MATH(47,10)	40
MOD	=MOD(ROW(),2)=0	TRUE on even rows (for banding rules)
PMT	=PMT(10%/12,36,-500000)	16133.59

PMT answers a question every business asks: a ₹5,00,000 delivery-van loan at 10% a year over 36 months costs ₹16,133.59 a month. The rate and the number of periods must use the same unit (monthly rate, months). Its relatives FV, PV, NPV, IRR, and XNPV handle savings, investment, and project decisions, and work the same way in Google Sheets.

Chapter 21 explains what medians, percentiles, and modes mean statistically and when each is the right summary.

Checking values: ISNUMBER, ISTEXT, ISBLANK, ISERROR, TYPE

Chapter 10 used ISNUMBER and ISTEXT to see what a cell holds. They're also building blocks: =SUMPRODUCT(--ISTEXT(Sales!E2:E331)) counts quantities stored as text (0 here), a one-cell data quality check. The -- turns TRUE and FALSE into 1 and 0. ISBLANK is TRUE only for truly empty cells, not for a formula returning ""; test =A2="" when you mean "looks empty".

Interview extra point. A common live test is "count the customers whose revenue is above average, by segment, without a pivot table". Say the plan before typing: a customer list with UNIQUE or a copied list, revenue with SUMIFS, segment with a lookup, then COUNTIFS with a criterion joined to AVERAGE. Naming the functions and the order shows structured thinking even if the clock runs out.

11.4 Lookups in depth

Chapter 10 used XLOOKUP for one job: fetch a customer's name for an exact code. Real work needs more: a whole row of details at once, the right band from a price or rebate table, the most recent record, or a value where a row meets a column. Figure 11.2 compares the three lookup styles you'll meet.

Customers sheet (rows 13–16 shown)

Question: what is the city for customer code 0014?

A customer_code	B customer_name	C city	D segment
0012	Fresh Bowl Kitchens	Bengaluru	Hospitality
0013	Om Sai Provisions	Nashik	Retail
0014	Deccan Packaging	Hyderabad	Wholesale
0015	Spice Route Restaurants	Kolkata	Hospitality

lookup column

return column

XLOOKUP	<code>=XLOOKUP("0014", A2:A25, C2:C25)</code> Point at the lookup column and the return column. Default: exact match.	→ Hyderabad
INDEX / MATCH	<code>=INDEX(C2:C25, MATCH("0014", A2:A25, 0))</code> MATCH finds the position (14); INDEX returns item 14. Works in every version.	→ Hyderabad
VLOOKUP	<code>=VLOOKUP("0014", A2:D25, 3, FALSE)</code> Counts columns (3). Insert a column inside A:D and it silently returns the wrong one.	→ Hyderabad

Figure 11.2 — Three ways to find the city for customer 0014. `XLOOKUP` and `INDEX / MATCH` point at the columns; `VLOOKUP` counts them, which is why it breaks when columns are inserted.

XLOOKUP, all its arguments

```
=XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])
```

Argument	Values	Meaning
if_not_found	any value	What to show when nothing matches (default #N/A)
match_mode	0 (default)	Exact match
	-1	Exact match, or the next smaller value
	1	Exact match, or the next larger value
	2	Wildcards: *, ? (Excel; Sheets uses 2 for wildcards too)
search_mode	1 (default)	Search from first to last
	-1	Search from last to first
	2, -2	Binary search on sorted data (fast, but wrong if the data isn't sorted)

Everything below works in Microsoft 365, Excel 2021 or later, Excel for the web, and Google Sheets.

Return several columns at once. "Show name, city, and segment for customer 0014."

```
=XLOOKUP("0014", Customers!A2:A25, Customers!B2:D25)
```

The return array is three columns wide, so the result is three cells: **Deccan Packaging · Hyderabad · Wholesale**. In Excel it spills across (section 11.6); in Google Sheets it fills the same three cells.

The most recent match. "When did Sharma Hardware last order?" The Sales table is sorted by date, so searching from the bottom finds the latest line:

```
=XLOOKUP("0001", Sales[customer_code], Sales[order_date], "never", 0, -1)
```

The result is **2025-12-05** (order 10161). With the default `search_mode` of 1, the same formula returns the first order, 2025-01-24 (order 10007). Home Plus (0016) placed no orders in 2025, so its lookup returns "never".

Watch out: "last" means last in the list. `search_mode -1` finds the last matching row, not the latest date. If the data isn't sorted by date, sort it first, or use `MAXIFS(Sales[order_date], Sales[customer_code], "0001")` (section 10.7), which finds the latest date regardless of order.

Approximate match: bands and tiers

Many business rules are **bands**: commission rates by sales, freight by order value, tax slabs by income. Riverstone's (illustrative) annual volume rebate works like this, on the **RebateBands** sheet:

annual_revenue_from	rebate_pct	band
0	0	Standard
250,000	1	Silver
400,000	2	Gold

A customer with ₹502,775 of revenue is in the Gold band, because ₹502,775 is at least ₹400,000. The lookup must find the **largest threshold that is less than or equal to** the value: exact, or the next smaller.

```
=XLOOKUP(502775, RebateBands!A2:A4, RebateBands!C2:C4, , -1)
```

The result is **Gold**. Western Logistics, with ₹248,727.50, gets **Standard**: it's ₹1,272.50 short of the Silver threshold.

Applied to all 23 customers who ordered in 2025, 2 are Gold (Sharma Hardware and Harbour Traders), 6 are Silver, and 15 are Standard, and the rebates total **₹37,642.03**.

Watch out: blank if_not_found. In `XLOOKUP(value, A, C, , -1)` the two commas leave `if_not_found` empty so that `-1` lands in the `match_mode` position. Count your commas: `XLOOKUP(value, A, C, -1)` would treat `-1` as the "not found" value and do an exact match.

INDEX and MATCH

Before `XLOOKUP`, analysts combined two functions, and you'll meet this pattern in almost every inherited workbook. It also works in every Excel version.

- `MATCH(value, range, 0)` returns the **position** of a value in a one-column range. `=MATCH("0014", Customers!A2:A25, 0)` returns **14**: 0014 is the 14th code in the list.
- `INDEX(range, position)` returns the item at that position. `=INDEX(Customers!C2:C25, 14)` returns **Hyderabad**.

Put them together:

```
=INDEX(Customers!C2:C25, MATCH("0014", Customers!A2:A25, 0))
```

`MATCH`'s last argument is the match type: `0` for exact, `1` for "largest value less than or equal to" (the range must be sorted ascending), and `-1` for "smallest value greater than or equal to" (sorted descending). The rebate band with `INDEX / MATCH`:

```
=INDEX(RebateBands!C2:C4, MATCH(502775, RebateBands!A2:A4, 1)) → Gold
```

Two-way lookups

"What did Wholesale customers buy in October?" When you have a summary grid with months down the side and segments across the top, you need the value where a row meets a column. With **INDEX** and two **MATCH** es:

```
=INDEX(B5:D16, MATCH(DATE(2025,10,1), A5:A16, 0), MATCH("Wholesale", B4:D4, 0))
```

The first **MATCH** finds the row (October is the 10th month), the second finds the column (Wholesale is the 3rd segment), and **INDEX** returns the cell: **₹207,827**. The **XLOOKUP** version nests one lookup inside another: the inner one returns October's whole row, and the outer one picks the Wholesale column from it.

```
=XLOOKUP("Wholesale", B4:D4, XLOOKUP(DATE(2025,10,1), A5:A16, B5:D16))
```

XMATCH: a better MATCH

XMATCH(value, lookup_array, [match_mode], [search_mode]) is **MATCH** with **XLOOKUP**'s options: exact match by default, wildcards, and searching from the end. **=XMATCH("0014", Customers!A2:A25)** returns **14**. Use it inside **INDEX** exactly as you'd use **MATCH**. (Microsoft 365 and Excel 2021 or later; Google Sheets has **XMATCH** too.)

Lookups with more than one condition

"What was the first order in which Harbour Traders bought Industrial Crates?" No single column identifies the row; two conditions do. Multiply the tests, which gives 1 only where both are true, and look up the 1:

```
=XLOOKUP(1, (Sales!M2:M331="Harbour Traders")*(Sales!P2:P331="Industrial Crate"), Sales!A2:A331, "none")
```

The result is **order 10057** (27 May 2025). The same idea with **INDEX / MATCH**, for any Excel version:

```
=INDEX(Sales!A2:A331, MATCH(1, (Sales!M2:M331="Harbour Traders")*(Sales!P2:P331="Industrial Crate"), 0))
```

In Excel 2019 and older, confirm it with **Ctrl+Shift+Enter** so Excel treats it as an array formula (curly brackets appear around it). In Microsoft 365 and Google Sheets, press Enter; in Sheets, wrap the multiplication in **ARRAYFORMULA** if it returns an error. Harbour Traders bought crates in 7 lines, which **COUNTIFS** confirms.

The older alternative is a **helper key**: a column that joins the fields, such as **=M2&"|"&P2**, and an ordinary lookup on **"Harbour Traders|Industrial Crate"**. It's fast and simple to audit, at the cost of an extra column.

Wildcard and partial-match lookups

"What's the code for the customer with 'Bay' in its name?"

```
=XLOOKUP("*Bay*", Customers!B2:B25, Customers!A2:A25, "none", 2)
```

match_mode 2 switches on wildcards, and the result is **0008** (Blue Bay Cafe). **INDEX / MATCH** supports wildcards with match type 0: **MATCH("*Bay*", Customers!B2:B25, 0)**. Partial matches return the **first** match, so check that the pattern is unique: **COUNTIF(Customers!B2:B25, "*Bay*")** should be 1.

Case-sensitive lookups

Lookups ignore case: "metro mart" finds Metro Mart . When case matters (product codes such as ab12 and AB12), use EXACT inside the multiply-and-match pattern:

```
=XLOOKUP(TRUE, EXACT(Customers!A2:A25, "0014"), Customers!B2:B25, "none")
```

Returning a whole row or column

INDEX with a row or column number of 0 returns the entire column or row, which you can then sum, average, or chart. With the segment-by-quarter grid from section 11.2 (segments in F16:F18 , quarters in G15:J15 , values in G16:J18):

```
=SUM(INDEX(G16:J18, 0, MATCH("Q2", G15:J15, 0))) → 726568.25  
=SUM(INDEX(G16:J18, MATCH("Wholesale", F16:F18, 0), 0)) → 1702658.5
```

The first adds Q2's column, the second adds Wholesale's row. XLOOKUP does the same when the return array is a block: =SUM(XLOOKUP("Q2", G15:J15, G16:J18)) .

The last match, in any version

Before XLOOKUP 's search_mode , analysts used a trick with the old LOOKUP function:

```
=LOOKUP(2, 1/(Sales!C2:C331="0001"), Sales!A2:A331) → 10161
```

1/(test) gives 1 where the test is true and #DIV/0! elsewhere. LOOKUP searches for 2, can't find it, ignores the errors, and returns the value for the **last** 1. You'll meet it in older workbooks; in new ones, use XLOOKUP with search_mode -1 .

INDIRECT and OFFSET: references built by formulas

Two functions build references while the formula runs. They're useful, occasionally essential, and the source of many slow and fragile workbooks.

INDIRECT(text) turns text into a reference. The messy month-end workbook has one sheet per month, named Jan , Feb , ... with the month names in A4:A15 of its Summary sheet. One formula, filled down, can total every month sheet:

```
=SUMIFS(INDIRECT("'"&A4&"'!J2:J100"), INDIRECT("'"&A4&"'!H2:H100"), "<>Cancelled")
```

It returns the correct valid total for every month in which the pasted data is correct, including October's full ₹681,070.75 that the hand-built formula missed. (December still shows ₹486,378.50, because INDIRECT can't know that five rows were pasted twice.)

OFFSET(start, rows, cols, [height], [width]) returns a range a given distance from a starting cell. "Sum the last three months that have data." With monthly revenue in G2:G13 :

```
=SUM(OFFSET(G2, COUNT(G2:G13)-3, 0, 3, 1)) → 1754302.25
```

COUNT finds 12 months, so the range starts 9 rows below G2 and is 3 rows tall: October to December.

Watch out: volatile and invisible. `INDIRECT` and `OFFSET` are **volatile**: Excel recalculates them after every change anywhere in the workbook, which slows large files. Their references are text, so renaming a sheet breaks `INDIRECT` silently, and **Trace Precedents** can't follow them. Prefer tables, `INDEX` (`=SUM(INDEX(G2:G13,COUNT(G2:G13)-2):INDEX(G2:G13,COUNT(G2:G13)))` does the same job non-volatily), `TAKE` (section 11.6), or Power Query's folder combine (section 11.7).

VLOOKUP and HLOOKUP: recognize, then replace

`VLOOKUP(value, table, column_number, FALSE)` looks in the **first** column of a table and returns the column at position `column_number`. `HLOOKUP` does the same sideways. Three traps:

1. **The fourth argument defaults to approximate.** Leave out `FALSE` and an unsorted list returns plausible wrong answers.
2. **The column number is a count.** Insert a column inside the table and `3` now points at a different column, with no error.
3. **It can only look right.** The lookup column must be the first column of the table.

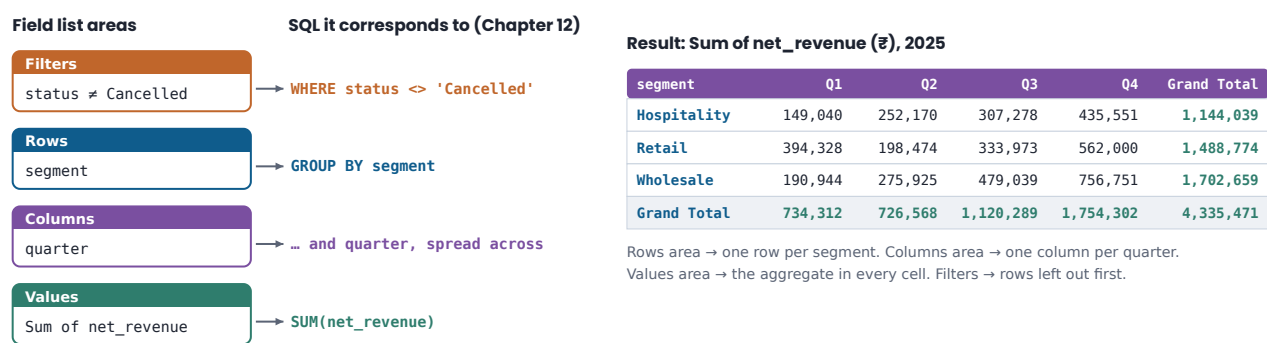
For bands, `VLOOKUP(value, RebateBands!A2:C4, 3, TRUE)` is legitimate and returns Gold for ₹502,775, because that's a deliberate approximate match on a sorted table. For everything else, use `XL00KUP` or `INDEX / MATCH`.

SQL link. An exact lookup is a `LEFT JOIN` (Chapter 12): every order line keeps its row, and a missing customer gives NULL, the database's version of "not found". A band lookup is a join on a range condition, such as `revenue >= threshold`, which Chapter 13's patterns handle with window functions.

Interview extra point. Asked "VLOOKUP or INDEX / MATCH?", say what breaks: `VLOOKUP` depends on a column count and only looks right; `INDEX / MATCH` and `XL00KUP` point at the columns directly. Then mention approximate matching for bands. That answer shows you've maintained real workbooks, not only learned syntax.

11.5 Pivot tables

A **pivot table** summarizes a table by dragging fields into four areas. It's the fastest way to answer "how much, by what?" and it's the single most used analysis feature in business spreadsheets. Figure 11.3 shows how its areas map to the SQL you'll write in Chapter 12.



The grand total reconciles to Chapter 10's tracker and Chapter 13's query: ₹4,335,471.

Figure 11.3 – A pivot table's areas and the SQL they correspond to. The result shows 2025 net revenue by segment and quarter, excluding cancelled orders.

Build your first pivot table

"How much net revenue did each segment bring in, each quarter?"

Excel

1. Click inside the Sales table. **Insert** → **PivotTable** → **From Table/Range**. Check the range says Sales, choose **New Worksheet**, and click **OK**.
2. In the **PivotTable Fields** pane, drag segment to **Rows**, quarter to **Columns**, and net_revenue to **Values**. Excel names it Sum of net_revenue.
3. Drag status to **Filters**. In the filter cell above the pivot, click the arrow, tick **Select Multiple Items**, and clear **Cancelled**.
4. Right-click any number, **Number Format...**, choose **Number** with a thousands separator and 0 decimals.

Google Sheets

1. Click inside the Sales data. **Insert** → **Pivot table**, confirm the range, choose **New sheet**, and click **Create**.
2. In the **Pivot table editor**, next to **Rows** click **Add** and choose segment; next to **Columns** add quarter; next to **Values** add net_revenue (**Summarize by: SUM**).
3. Next to **Filters**, add status, open **Status**, and clear **Cancelled** (or **Filter by condition** → **Text does not contain** → **Cancelled**).

Both give the table in Figure 11.3:

segment	Q1	Q2	Q3	Q4	Grand Total
Hospitality	149,040.00	252,170.00	307,277.50	435,551.25	1,144,038.75
Retail	394,327.50	198,473.75	333,972.50	562,000.00	1,488,773.75
Wholesale	190,944.00	275,924.50	479,039.00	756,751.00	1,702,658.50
Grand Total	734,311.50	726,568.25	1,120,289.00	1,754,302.25	4,335,471.00

The grand total is ₹4,335,471, the same as Chapter 10's tracker. ✓ What would have taken twelve SUMIFS formulas took four drags. Retail led in Q1; Wholesale grew every quarter and more than doubled from Q2 to Q4.

SQL link. This pivot is `SELECT segment, quarter, SUM(net_revenue) FROM sales WHERE status <> 'Cancelled' GROUP BY segment, quarter`, with the quarters spread into columns. Chapter 12 builds the same five-column shape with `CASE` expressions; the pivot does the spreading for you.

Summaries other than sum

Values don't have to be sums. **Excel:** right-click a value → **Summarize Values By** → **Count, Average, Max, Min**. **Sheets:** in the editor, change **Summarize by**.

Watch out: counting lines is not counting orders. Put `order_id` in Values and choose **Count**, and Wholesale shows **105**: that's order lines. Riverstone received **55** Wholesale orders. For distinct counts, **Excel:** when creating the pivot, tick **Add this data to the Data Model**, then **Summarize Values By** → **More Options** → **Distinct Count**. **Sheets:** **Summarize by: COUNTUNIQUE**. Distinct orders by segment: Hospitality 59, Retail 59, Wholesale 55, which add to 173. ✓

Show values as a percentage or a difference

"What share of revenue does each segment contribute?" Put `net_revenue` in Values a second time, then:

- **Excel:** right-click it → **Show Values As** → **% of Grand Total**.
- **Sheets:** in the value's settings, **Show as** → **% of grand total**.

Wholesale **39.3%**, Retail **34.3%**, Hospitality **26.4%**.

"How did each month change from the one before?" With `month_start` in Rows (grouped by month, below), **Excel:** **Show Values As** → **Difference From**, **Base field:** `month_start`, **Base item:** **(previous)**. June shows **-142,430.75** (₹186,928 against May's ₹329,358.75). Google Sheets' **Show as** offers only percentages of row, column, and grand total, so in Sheets add a formula beside the pivot or use Chapter 10's tracker for month-on-month changes.

Grouping dates

Pivot tables can group a date field into months, quarters, or years without helper columns.

- **Excel:** put `order_date` in Rows. Recent versions group it automatically; otherwise right-click a date → **Group...** → select **Months** and **Quarters** (and **Years** if the data spans more than one).
- **Sheets:** put `order_date` in Rows, right-click a date in the pivot → **Create pivot date group** → **Year-Month** or **Quarter**.

The practice table also has `month_start` and `quarter` columns, which are more predictable when a workbook will be used by other people or in both apps.

Sort, filter, and top N

"Which three products brought in the most revenue?" Put `product_name` in Rows and `net_revenue` in Values, then:

- **Excel:** click the Row Labels arrow → **Value Filters** → **Top 10...** → **Top 3 Items by Sum of net_revenue**. Sort with **More Sort Options** → **Descending by Sum of net_revenue**.
- **Sheets:** under Rows, set **Order: Descending** and **Sort by: SUM of net_revenue**; read the top three.

The top three: **Storage Box 25L ₹908,212.50**, **Storage Box 10L ₹860,946.00**, and **Industrial Crate ₹795,830.00**. The Garden Chair sold only ₹33,637.50 all year.

Slicers and timelines

A **slicer** is a set of filter buttons that sits beside a pivot table, which managers find far friendlier than filter dropdowns.

- **Excel:** click the pivot → **PivotTable Analyze** → **Insert Slicer** → tick `segment` and `sales_rep`. For dates, **PivotTable Analyze** → **Insert Timeline** → `order_date`. One slicer can control several pivots: right-click it → **Report Connections**.
- **Sheets:** **Data** → **Add a slicer**, choose the data range and the column. Sheets slicers filter every pivot table and chart built on that range.

"How did Rahul Mehta's quarters look?" With a rep-by-quarter pivot, click **Rahul Mehta** on the slicer: Q1 ₹278,630, Q2 ₹193,720, Q3 ₹338,761.50, Q4 **₹682,889**, the highest Q4 of any rep.

Refresh, and the traps that come with it

- **Excel pivot tables don't update by themselves.** After the data changes, click **PivotTable Analyze** → **Refresh** (Alt+F5) or **Data** → **Refresh All** (Ctrl+Alt+F5). To refresh every time the file opens: **PivotTable Options** → **Data** → **Refresh data when opening the file**.
- **Build on a table, not a range.** A pivot built on `A1:S331` won't see row 332. A pivot built on the `Sales` table sees every row the table grows to. If a pivot was built on a range, fix it with **PivotTable Analyze** → **Change Data Source**.
- **Google Sheets pivot tables update automatically** when the source data changes, but their range is fixed too; use an open-ended range such as `Sales!A1:S` or a table.
- **GETPIVOTDATA**. In Excel, clicking a pivot cell while writing a formula inserts `=GETPIVOTDATA(...)` instead of a cell address. It's robust when the pivot's layout changes but can't be copied down like a normal reference. Type the address instead, or turn it off: **PivotTable Analyze** → **Options** → **Generate GetPivotData**.

Watch out: a pivot built before the rule was agreed. If the `status` filter is added later, every number in the pivot changes. Put the filter (or a `valid` column) in from the start, and write the rule in the sheet's title, as in Chapter 10.

GETPIVOTDATA: reading numbers out of a pivot table

A report page often needs a handful of numbers from a pivot, placed in a fixed layout (a KPI panel, a board-pack table). Typing `=C7` breaks as soon as the pivot is rearranged or a new segment appears. `GETPIVOTDATA` asks the pivot for a value by **field names and items** instead of by position:

```
=GETPIVOTDATA("net_revenue", $A$3, "segment", "Wholesale", "quarter", "Q4") → 756751
```

Argument	Here	Meaning
<code>data_field</code>	<code>"net_revenue"</code>	The value field (the name of the source field; <code>"Sum of net_revenue"</code> also works)
<code>pivot_table</code>	<code>\$A\$3</code>	Any cell inside the pivot table

Argument	Here	Meaning
field1, item1	"segment", "Wholesale"	Which row item
field2, item2	"quarter", "Q4"	Which column item

Leave out a field to get its total: `=GETPIVOTDATA("net_revenue", $A$3, "segment", "Retail")` returns **1,488,773.75**, Retail's grand total across quarters, and `=GETPIVOTDATA("net_revenue", $A$3)` returns the grand total, 4,335,471.

Make it dynamic. Replace the typed items with cells, and one formula fills a whole report grid:

```
C6: =GETPIVOTDATA("net_revenue", Pivot!$A$3, "segment", $B6, "quarter", C$5)
```

With segments in `B6:B8` and quarters in `C5:F5`, fill it across and down. The report now keeps its own layout and formatting, while the pivot underneath can be refreshed, filtered with slicers, or rearranged.

How to get the formula without typing it: in Excel, type `=` and **click a value cell inside the pivot**. Excel writes the `GETPIVOTDATA` for you (if **PivotTable Analyze** → **Options** → **Generate GetPivotData** is ticked). Then replace the items with cell references.

Watch out: GETPIVOTDATA returns #REF! when the item isn't visible. If a slicer hides Wholesale, or the pivot no longer shows quarters, the formula can't find the cell and returns `#REF!`. Wrap it with `IFERROR(..., 0)` only when a missing item really means zero, and keep the fields it needs in the pivot. Google Sheets has `GETPIVOTDATA` too, with the same idea and argument order: `=GETPIVOTDATA("SUM of net_revenue", A3, "segment", "Wholesale")`.

For pivots built on the Data Model (section 11.8), Excel writes `CUBEVALUE` formulas instead, which do the same job with a different syntax; **PivotTable Analyze** → **OLAP Tools** → **Convert to Formulas** turns a whole model pivot into editable `CUBEVALUE` cells.

More ways to show values

Right-click a value → **Show Values As** (Excel) offers many calculations on top of the sum:

Option	Riverstone example	Result
% of Grand Total	Wholesale	39.3%
% of Row Total	Wholesale, Q4 (of Wholesale's year)	44.4%
% of Column Total	Wholesale's share of Q4	43.1%
Difference From (previous)	June vs May	−142,430.75
% Difference From (previous)	June vs May	−43.2%
Running Total In (month)	September, year to date	2,581,168.75
Rank Largest to Smallest (sales_rep)	Rahul Mehta, full year	1

Every segment earned more than a third of its year in Q4 (Hospitality 38.1%, Retail 37.7%, Wholesale 44.4%), which **% of Row Total** shows at a glance. Google Sheets offers **% of row**, **% of column**, and **% of grand total**.

Calculated fields, and why ratios need care

A **calculated field** adds a formula based on other fields' **totals**. "What price did Riverstone actually realize per unit, after discounts?" **Excel: PivotTable Analyze → Fields, Items & Sets → Calculated Field**, name `realized_price`, formula = `net_revenue / quantity`. **Sheets:** under **Values, Add → Calculated field**, formula `=SUM(net_revenue)/SUM(quantity)`.

With `product_name` in Rows, Industrial Crate shows ₹1,273.33 a unit against a list price of ₹1,400: 9.0% below list, the bulk discounts from section 11.8's margin table in another form. A calculated field divides the **sums**, which is right for a ratio. Averaging a per-line price column instead would weight a 5-unit line the same as a 45-unit line.

Grouping numbers into bands

Put `net_revenue` in **Rows** as well as Values, right-click a row value → **Group...**, **Starting at 0, Ending at 60000, By 10000**. Excel groups lines by value band:

Line value band	Lines	Net revenue (₹)
0-9,999	158	912,950.50
10,000-19,999	106	1,495,979.50
20,000-29,999	36	874,648.50
30,000-39,999	14	471,117.50
40,000-49,999	9	421,035.00
50,000-59,999	3	159,740.00

Almost half the lines (158 of 326) are under ₹10,000, but they bring in only 21% of revenue. That's a distribution, and Chapter 21 turns it into a proper histogram.

Report filter pages

Excel: PivotTable Analyze → Options (dropdown) → Show Report Filter Pages. With `sales_rep` in Filters, Excel copies the pivot onto a new sheet for **each rep**, filtered to that rep. It's a two-click way to produce one page per region, branch, or manager.

Pivot charts and layout

Excel: PivotTable Analyze → PivotChart. The chart follows the pivot's filters and slicers. **Sheets:** select the pivot table and **Insert → Chart**; it updates with the pivot.

For reports that others will copy, change the layout to a plain table: **Excel: Design → Report Layout → Show in Tabular Form** and **Repeat All Item Labels**; turn off **Subtotals** if you don't need them.

11.6 Dynamic arrays: formulas that return whole tables

Every formula in Chapter 10 returned one value into one cell. A **dynamic array** formula returns a whole list or table from a single cell, and the results **spill** into the cells below and to the right. When the source data changes, the result grows or shrinks by itself. Figure 11.4 shows the idea.

The formula lives in one cell...

F2: =SORT(UNIQUE(Sales[customer_name]))

F
1 customer (23 names)
2 Blue Bay Cafe
3 City Needs Store
4 Coastal Foods
5 Deccan Packaging
6 Evergreen Mart
7 ... 18 more rows
8

...and a blocked spill shows #SPILL!

Someone typed a note in F4

F
1 customer (23 names)
2 #SPILL!
3
4 check later
5
6
7
8

The dashed range is the spill range. Only F2 holds a formula; the other cells show its results and can't be typed into. Refer to the whole result as F2# (Excel); it grows or shrinks with the data. Clear the blocking cell and it spills again. Google Sheets spills the same way; a blocked result shows #REF! (array not expanded).

Figure 11.4 – One formula, many results. The dashed outline is the spill range; any value typed inside it blocks the spill.

Dynamic arrays are in Microsoft 365, Excel 2021 and later, Excel for the web, and Google Sheets. Excel 2019 and older can open the file but show only the first result, or an older array-formula version.

UNIQUE, SORT, and SORTBY

"List every customer who ordered in 2025, alphabetically." In an empty column:

```
=SORT(UNIQUE(Sales[customer_name]))
```

23 names spill down, from Blue Bay Cafe, City Needs Store, Coastal Foods, Deccan Packaging, Evergreen Mart ... Riverstone has 24 customers, but Home Plus didn't order in 2025, so it isn't in the sales data. Count them with `=COUNTA(UNIQUE(Sales[customer_name]))` → 23, or `=ROWS(...)`.

`SORT` differs between the apps:

	Excel	Google Sheets
Syntax	<code>SORT(array, [sort_index], [sort_order], [by_col])</code>	<code>SORT(range, sort_column, is_ascending, ...)</code>
Descending by column 2	<code>=SORT(A2:B24, 2, -1)</code>	<code>=SORT(A2:B24, 2, FALSE)</code>
Sort by another range	<code>SORTBY(array, by_array, -1)</code>	<code>SORT(range, by_range, FALSE)</code>

"Rank customers by revenue." Put the unique names in F2 (the formula above) and their revenue next to them with `SUMIFS`, which accepts a spilled range as its criteria:

```
G2: =SUMIFS(Sales[net_revenue], Sales[customer_name], F2#, Sales[status], "<>Cancelled")
```

In Excel, `F2#` means "the whole spill range that starts at F2", so `G2` spills 23 totals to match. Then `=SORTBY(F2#, G2#, -1)` lists customers from largest to smallest: Sharma Hardware ₹502,775, Harbour Traders ₹412,680.50, Northgate Distributors ₹353,088.50, Green Leaf Hotels ₹331,388.75, Metro Mart ₹329,297.50 ... Google Sheets has no `#` operator; refer to a range that's big enough, such as `F2:F24`, or wrap the formula in `ARRAYFORMULA` (section 11.10).

FILTER

"Show November's Wholesale order lines."

```
=FILTER(Sales[[order_id]:[net_revenue]], (Sales[month_start]=DATE(2025,11,1)) * (Sales[segment]="Wholesale"), "no rows")
```

15 rows spill out, starting with order 10139 from Harbour Traders on 6 November. Their net revenue totals **₹343,685.50**, matching Chapter 10's `SUMIFS` . ✓

How it works:

- The second argument is an **include** test that's TRUE or FALSE for every row. `(test1) * (test2)` means **AND** ($\text{TRUE} \times \text{TRUE} = 1$); `(test1) + (test2)` means **OR**.
- The third argument is what to show when nothing matches. Without it, an empty result is a `#CALC!` error in Excel.
- `Sales[[order_id]:[net_revenue]]` is a table reference for the columns from `order_id` to `net_revenue` .

Google Sheets writes each condition as its own argument, and has no "if empty" argument:

```
=FILTER(Sales!A2:J331, Sales!K2:K331=DATE(2025,11,1), Sales!N2:N331="Wholesale")
```

`FILTER` 's output is live. Wrap it in `SUM`, `COUNTA`, or `SORT`: `=SUM(FILTER(Sales[net_revenue], Sales[segment]="Wholesale", 0))` works like `SUMIFS`, but `FILTER` can use conditions `SUMIFS` can't, such as `MONTH(Sales[order_date])=11` or `ISNUMBER(SEARCH("Box", Sales[product_name]))` .

SEQUENCE and LET

`SEQUENCE(rows, [columns], [start], [step])` generates numbers. Twelve month starts for 2025, in one formula:

```
=DATE(2025, SEQUENCE(12), 1)
```

`LET` names parts of a formula so it's readable and calculates each part once:

```
=LET(
  valid, Sales[status]<>"Cancelled",
  rev, FILTER(Sales[net_revenue], valid),
  margin, FILTER(Sales[gross_margin], valid),
  SUM(margin) / SUM(rev)
)
```

The result is **0.262**: Riverstone's 2025 gross margin was **26.2%** of net revenue (₹1,137,221 on ₹4,335,471). Both apps support `LET` .

Shaping arrays: TAKE, DROP, CHOOSECOLS, VSTACK, HSTACK, TOCOL

Microsoft 365 has functions that cut, pick, and stack arrays, so a report table can be assembled in one formula.

Function	Example	What it does
<code>TAKE(array, rows, [cols])</code>	<code>=TAKE(SORTBY(F2#, G2#, -1), 3)</code>	First 3 rows (negative numbers take from the end)
<code>DROP(array, rows, [cols])</code>	<code>=DROP(A1:S331, 1)</code>	Everything except the first row
<code>CHOOSECOLS(array, n, ...)</code>	<code>=CHOOSECOLS(Sales!A2:S331, 1, 13, 10)</code>	Columns 1, 13, 10 in that order
<code>CHOOSEROWS(array, n, ...)</code>	<code>=CHOOSEROWS(G2:G13, 10, 11, 12)</code>	Rows 10-12
<code>VSTACK(a, b, ...)</code>	<code>=VSTACK(Jan!A2:I16, Feb!A2:I21)</code>	Stack ranges on top of each other

Function	Example	What it does
HSTACK(a, b, ...)	=HSTACK(F2#, G2#)	Put arrays side by side
TOCOL(array, [ignore])	=TOCOL(I2:I331, 1)	One column, skipping blanks
EXPAND , WRAPROWS	=WRAPROWS(TOCOL(list), 4)	Reshape a list into a grid

"A top-3 customer table with names and revenue, in one cell."

```
=TAKE(SORTBY(HSTACK(F2#, G2#), G2#, -1), 3)
```

Result: Sharma Hardware ₹502,775.00, Harbour Traders ₹412,680.50, Northgate Distributors ₹353,088.50. Change the 3 to a cell reference and the reader chooses the length. Google Sheets has VSTACK , HSTACK , TOCOL , CHOOSECOLS , CHOOSEROWS , WRAPROWS , and EXPAND ; for TAKE , use ARRAY_CONSTRAIN or QUERY with limit .

LAMBDA: write your own functions

LAMBDA creates a function from a formula. Its last argument is the calculation; the ones before it are parameter names.

```
=LAMBDA(qty, price, disc, qty*price*(1-disc/100))(45, 750, 5) → 32062.5
```

Typed like that, it's a curiosity. Its value comes when you **name** it: **Formulas → Name Manager → New**, name NETREV , and in **Refers to** enter =LAMBDA(qty, price, disc, qty*price*(1-disc/100)) . Now every sheet in the workbook can use =NETREV(E2, F2, G2) , and the business rule lives in one place. Google Sheets has the same idea under **Data → Named functions**, where you define the formula and argument names.

MAP, BYROW, BYCOL, SCAN, REDUCE

These **helper functions** pass each item (or row, or column) of an array to a LAMBDA .

Function	Passes	Riverstone example	Result
MAP(array, LAMBDA(x, ...))	Each value	=MAP(E2:E331, F2:F331, G2:G331, NETREV)	330 net revenue values
BYROW(array, LAMBDA(row, ...))	Each row	=BYROW(G16:J18, LAMBDA(r, SUM(r)))	Segment totals: 1,144,038.75 · 1,488,773.75 · 1,702,658.50
BYCOL(array, LAMBDA(col, ...))	Each column	=BYCOL(G16:J18, LAMBDA(c, SUM(c)))	Quarter totals
SCAN(start, array, LAMBDA(acc, x, ...))	Each value, keeping a running result	=SCAN(0, G2:G13, LAMBDA(acc, x, acc+x))	Running total ending 4,335,471
REDUCE(start, array, LAMBDA(acc, x, ...))	Each value, returning only the final result	=REDUCE(0, G3:G13-G2:G12, LAMBDA(acc, x, MAX(acc, x)))	229,032.75 (the largest month-on-month rise, September)

SCAN is the formula version of a running balance, and the championship case at the end of this chapter uses exactly that idea to simulate stock levels day by day. Google Sheets has MAP , BYROW , BYCOL , SCAN , REDUCE , and MAKEARRAY too.

Watch out: clever isn't the same as clear. A single-cell `LET – LAMBDA – SCAN` formula can replace fifty helper cells, and nobody else in your team may be able to check it. For shared workbooks, prefer visible helper columns or name each step with `LET`, and keep the one-cell versions for places where speed matters more than handover, such as a timed competition.

GROUPBY and PIVOTBY (newer Microsoft 365 functions)

Microsoft 365 has two newer functions that build a pivot-style summary as a formula. They aren't in Google Sheets or in older perpetual versions of Excel (check Microsoft's `GROUPBY` help page for the current list), so use them only when every reader has them.

```
=GROUPBY(Sales[segment], Sales[net_revenue], SUM, , , , Sales[status]<>"Cancelled")
```

The result is a three-row summary with a total: Hospitality ₹1,144,038.75, Retail ₹1,488,773.75, Wholesale ₹1,702,658.50, total ₹4,335,471. The seventh argument is the filter; the empty arguments in between keep the defaults for headers, totals, and sorting. `PIVOTBY` adds a column field: `=PIVOTBY(Sales[segment], Sales[quarter], Sales[net_revenue], SUM, , , , , Sales[status]<>"Cancelled")` rebuilds Figure 11.3's grid.

Watch out: forgetting the filter. Without the filter argument, Retail shows ₹1,551,423.75, including cancelled order 10131's ₹37,850 and 10034's ₹24,800. A formula summary is only as correct as its rules, exactly like a pivot.

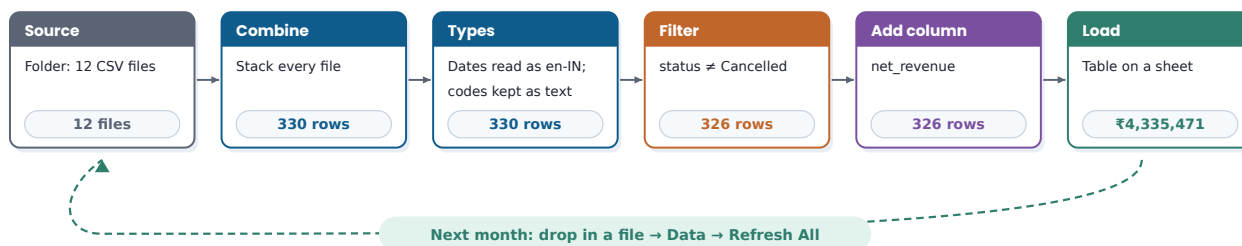
Living with spills

- **#SPILL!** (Excel) means something is in the way. Click the error's warning icon → **Select Obstructing Cells**, and clear them. Google Sheets shows **#REF!** with the message that the array result wasn't expanded because it would overwrite data.
- **You can't type inside a spill range** or delete one cell of it. Edit or delete the formula in its first cell.
- **Spills and tables don't mix in Excel.** A dynamic array formula can't spill inside an Excel table; put spilled results on a report sheet.
- **@** in a formula (Excel) means "one value, not the array". You'll see it when older workbooks are opened in Microsoft 365. It's safe to leave.

11.7 Power Query: record the cleanup once, refresh forever

Power Query is Excel's built-in tool for getting data from files, folders, databases, and the web, cleaning it with recorded steps, and loading the result into a table. It's the same engine as Power Query in Power BI (Chapter 16). You met it briefly in Chapter 10, where it imported a CSV with the right types. Here you'll use it to replace the month-end copy-paste routine completely. Figure 11.5 shows the query you'll build.

Applied Steps in the Power Query Editor, recorded once and replayed on every Refresh



Nothing here edits the CSV files. Each step is a recorded instruction; delete or edit a step and everything after it recalculates.

Figure 11.5 – The Riverstone query as applied steps. Each step is an instruction, not a change to the files; Refresh replays them on whatever files are in the folder.

Tool note: where Power Query runs. The full Power Query Editor is in Excel for Windows (Microsoft 365, Excel 2016 and later). Excel for Microsoft 365 for Mac includes the Power Query Editor with fewer data sources, and Excel for the web can refresh and edit some queries. Google Sheets has no Power Query; section 11.11 gives the alternatives. Features in this area change often, so check Microsoft's current "Power Query in Excel" help page for your platform.

Step 1: connect to the folder

The folder `monthly_exports/` holds twelve files, `sales_2025_01.csv` to `sales_2025_12.csv`, each with the same nine columns as Chapter 10's export (day-first dates, zero-padded codes). They have 15, 20, 18, 21, 29, 22, 29, 28, 40, 38, 35, and 35 lines: 330 in total.

1. Copy the folder somewhere stable, such as `C:\Riverstone\monthly_exports`.
2. In a new workbook: **Data → Get Data → From File → From Folder**, choose the folder, and click **Open**.
3. A preview lists the twelve files. Click **Combine → Combine & Transform Data**.
4. In **Combine Files**, choose the first file as the sample, check that the delimiter is **Comma**, and click **OK**.

The **Power Query Editor** opens with all 330 rows stacked, plus a `Source.Name` column showing which file each row came from. On the left, **Queries** lists your main query and some helper queries (`Transform Sample File` and others) that Excel created to apply the same steps to every file. On the right, **Applied Steps** lists everything done so far.

Step 2: fix the types (and undo Excel's guess)

Look at the last applied step: **Changed Type**. Power Query guessed the types, and it guessed `customer_code` is a whole number, which strips the zeros, exactly the problem from Chapter 10. It may also have read the dates using your computer's regional settings.

1. Click the **×** next to **Changed Type** to delete that step.
2. Select `customer_code`, `status`, and `sales_rep`; **Transform → Data Type → Text**.
3. Select `order_id`, `product_id`, and `quantity`; **Data Type → Whole Number**. Select `unit_price` and `discount_pct`; **Data Type → Decimal Number**.
4. Right-click `order_date` → **Change Type → Using Locale... → Date, English (India) → OK**.
5. Right-click `Source.Name` → **Remove** (or keep it as a record of the source file).

Watch out: the automatic "Changed Type" step comes back. Power Query adds a type-detection step after many operations, including combining files and promoting headers. Read every **Changed Type** step it creates. To stop it for new queries, **File → Options and settings → Query Options → Data Load → Type Detection → Never detect column types and headers for unstructured sources**.

Step 3: apply the business rule

Click the arrow on the `status` header, clear **Cancelled**, and click **OK**. The step **Filtered Rows** appears, and the row count at the bottom left drops to **326**.

Step 4: add the calculated column

Add Column → Custom Column. Name it `net_revenue` and enter:

```
[quantity] * [unit_price] * (1 - [discount_pct] / 100)
```

Then set its type to **Decimal Number**. Power Query's formula language is called **M**; column names go in square brackets, and the formula runs for every row, so there's nothing to fill down.

Step 5: bring in the customer segment with a merge

A **merge** is a lookup done once for the whole table (a join, in Chapter 12's terms).

1. First load the Customers list as its own query: in the practice workbook, click in the Customers data, **Data → From Table/Range**, set `customer_code` to **Text**, and **Home → Close & Load To... → Only Create Connection**.
2. Back in the sales query: **Home → Merge Queries**. Choose the Customers query, click `customer_code` in both tables, and choose **Join Kind: Left Outer (all from first, matching from second)**. The dialog reports how many rows matched: it should say 326 of 326.
3. A new column of nested tables appears. Click its expand icon, tick `customer_name` and `segment`, clear **Use original column name as prefix**, and click **OK**.

Watch out: a merge that matches fewer rows than you have. If the merge dialog says "The selection matches 0 of the 326 rows", the key columns have different types (text `0014` against the number 14). Fix the types in both queries before merging.

Step 6: load and check

Home → Close & Load. The query's result lands on a new sheet as a table, and **Data → Queries & Connections** lists the query with "326 rows loaded". Check it:

```
=SUM(RiverstoneSales[net_revenue]) → 4335471
```

Rename the query and its table `RiverstoneSales` in the Queries pane first. The total matches every other method in this book. ✓

Next month: refresh

This is the payoff. When January 2026's export arrives, save it into the same folder and click **Data → Refresh All**. Power Query replays every applied step on every file in the folder. Nobody pastes, extends ranges, or re-types a rule.

Try a refresh test now: move `sales_2025_12.csv` out of the folder and click **Refresh All**. The total drops to **₹3,895,647.50** (December's ₹439,823.50 is gone) and any pivot built on the table shows Q4 as ₹1,314,478.75 after its own refresh. Move the file back, refresh, and ₹4,335,471 returns.

Watch out: a new file with a different layout. If a month's export has an extra column, a renamed header, or a summary line at the bottom, the combined query can fail or silently put values in the wrong columns. Check the row count after every refresh (section 11.12), and ask the ERP team to keep the export layout fixed.

More Power Query moves you'll use

Unpivot: turning wide data into long data. The **TargetsWide** sheet has one row with a column per month, the way targets often arrive from finance. Pivot tables and joins need one row per month. Load it with **Data → From Table/Range**, select the `year` column, and **Transform → Unpivot Other Columns**. Twelve rows appear, with `Attribute` (the month) and `Value` (the target), totaling ₹4,240,000. Rename the columns `month` and `target`.

Other everyday steps: **Remove Duplicates**, **Split Column** (by delimiter), **Replace Values**, **Trim** and **Clean** (under **Transform → Format**), **Fill Down** for blank cells under a group heading, **Group By** (a pivot inside the query), and **Append Queries** to stack two queries with the same columns.

Parameters. Hard-coding `C:\Riverstone\monthly_exports` breaks when a colleague runs the file. **Home → Manage Parameters → New Parameter** creates a named value (such as `FolderPath`) that the Source step can use, so the path is changed in one place.

The M code. **Home → Advanced Editor** shows the whole query as M code. You don't need to write M by hand to use Power Query, but reading it helps you understand what a query does. The companion file `riverstone_sales.pq` holds a tidied version of this section's query, which you can paste into **Data → Get Data → From Other Sources → Blank Query → Advanced Editor** (change `FolderPath` first):

```

let
    FolderPath = "C:\Riverstone\monthly_exports",
    Source      = Folder.Files(FolderPath),
    CsvOnly     = Table.SelectRows(Source,
        each Text.Lower([Extension]) = ".csv"),
    Parsed      = Table.AddColumn(CsvOnly, "Data",
        each Table.PromoteHeaders(
            Csv.Document([Content],
                [Delimiter = ",", Encoding = 65001,
                QuoteStyle = QuoteStyle.Csv]),
            [PromoteAllScalars = true])),
    Combined    = Table.Combine(Parsed[Data]),
    Typed       = Table.TransformColumnTypes(Combined, {
        {"order_id", Int64.Type},
        {"customer_code", type text},
        {"product_id", Int64.Type},
        {"quantity", Int64.Type},
        {"unit_price", type number},
        {"discount_pct", type number},
        {"status", type text},
        {"sales_rep", type text}}),
    TypedDates  = Table.TransformColumnTypes(Typed,
        {"order_date", type date}}, "en-IN"),
    Valid       = Table.SelectRows(TypedDates,
        each [status] <> "Cancelled"),
    NetRevenue  = Table.AddColumn(Valid, "net_revenue",
        each [quantity] * [unit_price]
            * (1 - [discount_pct] / 100),
        type number)
in
    NetRevenue

```

Each line inside `let` is one applied step, named on the left and built from the step before it; `in` names the step whose result is loaded.

Loading options. **Home** → **Close & Load To...** offers **Table** (a sheet table), **PivotTable Report**, **Only Create Connection** (for helper queries), and **Add this data to the Data Model** (section 11.8). Large data (hundreds of thousands of rows) should go to the Data Model, not to a sheet.

Power Query in depth

The six steps above are the core. The tools below turn Power Query from "a better import" into the place where most of a monthly report's cleanup and shaping happens.

See the data before you change it: column profiling

In the Power Query Editor, tick **View** → **Column quality**, **Column distribution**, and **Column profile**. Then click the status bar text **Column profiling based on top 1000 rows** and switch it to **entire data set**, or the profile describes only a sample.

- **Column quality** shows the percentage of each column that is **Valid**, **Error**, or **Empty**. On the combined sales query, `sales_rep` is 6% empty: the 19 lines with no rep.
- **Column distribution** shows **distinct** and **unique** counts. `customer_code` has 23 distinct values and 0 unique ones (every customer ordered more than once).
- **Column profile** (for the selected column) shows statistics and a value distribution: `status` has 322 Delivered, 4 Cancelled, 3 Shipped, and 1 Pending line.

Profile every new source before building steps on it. Chapter 14 turns this into a full data-quality routine.

Conditional columns and columns from examples

Add Column → Conditional Column builds an if-then-else column without writing M. "Classify each line by size." New column name `size`; **If** `net_revenue` **is greater than or equal to** `30000` **Then** `Large`; **Else If** `net_revenue` **is greater than or equal to** `10000` **Then** `Medium`; **Else** `Small`. On the 326 valid lines: 158 Small, 142 Medium, 26 Large, the same bands as the `IFS` formula in section 11.3, now calculated once for every refresh.

Add Column → Column From Examples lets you type the result you want for a row or two, and Power Query writes the transformation. Type `Neha` next to `Neha Kulkarni` and it proposes **Text Before Delimiter**; accept it and check the formula it shows at the top before clicking **OK**.

Group By with several aggregations

Home → Group By → Advanced. "One row per customer with revenue, lines, orders, and first and last order dates." Group by `customer_name`, then add aggregations:

New column	Operation	Column
<code>revenue</code>	Sum	<code>net_revenue</code>
<code>lines</code>	Count Rows	—
<code>orders</code>	Count Distinct Rows	(select <code>order_id</code> first, see note)
<code>first_order</code>	Min	<code>order_date</code>
<code>last_order</code>	Max	<code>order_date</code>

Sharma Hardware's row: ₹502,775, 32 lines, 16 orders, first order 24 January, last 5 December.

Tool note: distinct counts in Group By. **Count Distinct Rows** counts distinct *whole rows*, not distinct values of one column. To count distinct orders, the simplest route is a second Group By: first group by `customer_name` and `order_id` (Count Rows), then group that result by `customer_name` (Count Rows again). In the Advanced Editor you can instead write `each List.Count(List.Distinct([order_id]))`.

Merge join kinds, including anti-joins

The **Join Kind** in **Merge Queries** decides which rows survive, exactly like SQL joins in Chapter 12:

Join kind	Keeps	Riverstone use	Result
Left Outer	All rows from the first table, matches from the second	Sales + Customers (add segment)	326 rows
Right Outer	All rows from the second, matches from the first	Rarely needed; swap the tables instead	—
Full Outer	All rows from both	Reconciling two lists	—
Inner	Only rows that match in both	Sales lines with a known customer	326 rows
Left Anti	Rows in the first with no match in the second	Customers with no 2025 sales	1 row: Home Plus
Right Anti	Rows in the second with no match in the first	Sales lines with an unknown customer code	0 rows

Anti-joins are the quiet heroes of reconciliation. "Which customers didn't buy this year?" Merge **Customers** (first) with **Sales** (second) on `customer_code`, **Left Anti**: one row, Home Plus. A **Right Anti** merge that returns rows means the sales data has codes the customer master doesn't know, which should stop a report until

someone explains them.

Fuzzy merge, briefly

Tick **Use fuzzy matching to perform the merge** in the merge dialog to match text that is similar rather than identical ("Metro Mart" and "Metro Mart Pvt Ltd"). **Fuzzy matching options** set the **Similarity threshold** (0 to 1; higher is stricter) and can ignore case and spaces. Fuzzy merges are useful and risky: always check the matches in a filtered view before trusting them. Chapter 14 uses them on Riverstone's messy customer export.

Append, pivot, and split into rows

- **Append Queries** stacks queries with the same columns (this year's query and last year's archive). Columns are matched **by name**; a column missing from one query is filled with nulls.
- **Transform → Pivot Column** is the reverse of unpivot: select `quarter`, choose **Values Column** `net_revenue`, **Advanced options → Aggregate Value Function: Sum**, and the segment-by-quarter grid appears inside the query.
- **Split Column → By Delimiter → Advanced options → Split into Rows** turns a cell holding `"101;105;107"` into three rows, one per product. It's the fix for lists typed into one cell.
- **Transform → Fill → Down** copies a value into the blank cells below it, for reports where a customer name appears only on its first line.

When a file breaks the rules: errors, extra columns, and footers

Open `pq_practice/sales_export_with_problems.csv` in Notepad. It's a five-line export with three problems real exports often have: an extra `remarks` column, a quantity typed as `55 pcs`, and a footer line `Total lines: 5`.

1. **Load it (Data → From Text/CSV → Transform Data)** and delete the automatic **Changed Type** step. Six rows appear: five lines and the footer.
2. **Remove the footer: Home → Remove Rows → Remove Bottom Rows → 1**. Five rows remain. (If footers vary, filter `order_id` to keep only values that are numbers, or use **Remove Rows → Remove Errors** after typing the column.)
3. **Set types**. Changing `quantity` to **Whole Number** turns `55 pcs` into an **Error** cell. **Home → Keep Rows → Keep Errors** shows the one bad row so you can see what went wrong; delete that step afterward.
4. **Fix instead of dropping**. Before the type step, **Transform → Extract → Text Before Delimiter** with a space, or a custom step: `Table.TransformColumns(Source, {"quantity", each Number.From(Text.Select(Text.From(_), {"0".."9"})), Int64.Type})`, which keeps only the digits. `55 pcs` becomes `55`.
5. **Handle columns you don't need: Home → Choose Columns** and keep the nine standard columns. In M, `Table.SelectColumns(Source, {...}, MissingField.UseNull)` also survives a month in which a column is **missing**.

With those steps, the five lines total **₹46,555**, the value of the December lines they were copied from. The same pattern (remove footers, extract digits, choose columns) makes the monthly folder query survive a badly behaved export.

In M, `try ... otherwise ...` catches errors in a custom column: `each try Number.From([quantity]) otherwise null` returns null instead of an error, so a single bad value doesn't stop the refresh. Pair it with a check (a count of nulls) so bad values are noticed.

Custom functions: the same steps for every file

When **Combine Files** runs, Excel builds a hidden **function** from the sample file's steps and applies it to each file. You can do this yourself for full control:

1. Build a query that loads **one** monthly file and cleans it (types, footer removal, `net_revenue`).
2. **Home** → **Manage Parameters** → **New Parameter**, `FilePath`, type **Text**, current value the file's path. Edit the Source step to use `FilePath` instead of the typed path.
3. Right-click the query → **Create Function...**, name it `fnReadExport`.
4. In the folder query, **Add Column** → **Invoke Custom Function** → **fnReadExport**, passing the folder path and file name joined together; then expand the result.

The custom function is easier to read and change than Excel's generated helper queries, and it can be reused for next year's folder.

Staging queries and the dependency view

Split big transformations into **staging** queries (load, type, clean) and **reporting** queries that start from them: right-click a query → **Reference** creates a new query whose first step is the other query's result. Set staging queries to **Only Create Connection** so they don't fill sheets. **View** → **Query Dependencies** draws how queries feed each other, which is the first thing to look at in an inherited workbook.

Query folding and performance

When the source is a **database** (Chapter 12), Power Query tries to translate your steps into one SQL query that the database runs, so only the result travels to Excel. This is **query folding**. Right-click a step: if **View Native Query** is available, the steps up to there are folding. Filters, column choices, joins, and grouping usually fold; custom M functions and some text operations stop folding, and every step after that runs on your computer.

Performance habits:

- **Filter rows and remove columns early**, right after the Source step.
- **Don't load helper queries** to sheets.
- **Prefer database views** (Chapter 12) for heavy joins rather than merging large tables in Power Query.
- **Avoid loading hundreds of thousands of rows to a sheet**; load them to the Data Model and pivot from there.
- **Turn off background data previews** for very large sources (**Query Options** → **Current Workbook** → **Data Load**).

When a refresh fails

Message or symptom	Usual cause	Fix
DataSource.Error: Could not find a part of the path	Folder moved or the file path is on someone else's computer	Use a parameter for the path; store data on a shared location
Expression.Error: The column 'X' of the table wasn't found	A header was renamed or removed in the source	Fix the source, or use <code>MissingField.UseNull</code> ; avoid steps that hard-code every column
DataFormat.Error: We couldn't convert to Number	Text in a numeric column (55 pcs , - , N/A)	Keep Errors to find it; clean before typing
Refresh works but the total is wrong	A file was duplicated in the folder, or a month is half-exported	Add <code>Source.Name</code> and count lines per file; compare with the ERP

Message or symptom	Usual cause	Fix
Formula.Firewall: Query references other queries or steps	Mixing data sources with different privacy levels in one step	File → Options and settings → Query Options → Privacy ; separate the sources into staging queries
Refresh is very slow	No folding, many merges on large tables, sheets loaded with helper queries	Filter early; load to the Data Model; move heavy joins to the database

Tool note: refresh settings. In **Data → Queries & Connections**, right-click a query → **Properties** to set **Refresh data when opening the file** or **Refresh every n minutes**. A query that refreshes only when someone opens the workbook is still manual; Chapter 20 covers scheduling refreshes with Power Automate, and Chapter 16 with the Power BI service.

11.8 Power Pivot and a first taste of DAX

A pivot table on one wide table works well, but real businesses have several related tables: order lines, customers, products, dates. Copying every customer and product column into the sales table (as the practice Sales sheet does) repeats data and grows quickly. **Power Pivot** lets Excel hold several tables in a **data model**, connect them with **relationships**, and calculate with **DAX** (Data Analysis Expressions), the same language Power BI uses (Chapter 16).

Tool note: availability. Power Pivot and the Data Model are in Excel for **Windows** (Microsoft 365 and Excel 2016 or later). They are **not** available in Excel for Mac, and Excel for the web can use existing models only in limited ways. Google Sheets has no equivalent. If you work on a Mac, read this section for the ideas; you'll practice DAX again in Chapter 16 (Power BI Desktop is also Windows-only, so Mac users typically run it in a Windows virtual machine or a cloud PC).

Turn it on and build the model

1. Enable the add-in if you don't see a **Power Pivot** tab: **File → Options → Add-ins → Manage: COM Add-ins → Go** → tick **Microsoft Power Pivot for Excel**.
2. Load the tables into the model. For the sales query from section 11.7: **Data → Queries & Connections**, right-click `RiverstoneSales` → **Load To...** → tick **Add this data to the Data Model**. Do the same for Customers and Products (from the practice workbook, via **Data → From Table/Range → Close & Load To... → Only Create Connection + Add this data to the Data Model**). Use the practice file's plain Sales columns (A to I) rather than the pre-joined ones, so the model does the joining.
3. Create a date table: **Power Pivot → Manage**, then in the Power Pivot window **Design → Date Table → New**. Excel adds a `Calendar` table covering the dates in the model, and marks it as the date table. (If Excel's automatic table spans more years than you want, filter it later in pivots.)
4. Create relationships: in the Power Pivot window, **Home → Diagram View**, and drag `customer_code` in the sales table onto `customer_code` in Customers, `product_id` onto `product_id` in Products, and `order_date` onto `Date` in Calendar. Or use **Data → Relationships → New** in Excel.

Figure 11.6 shows the result.

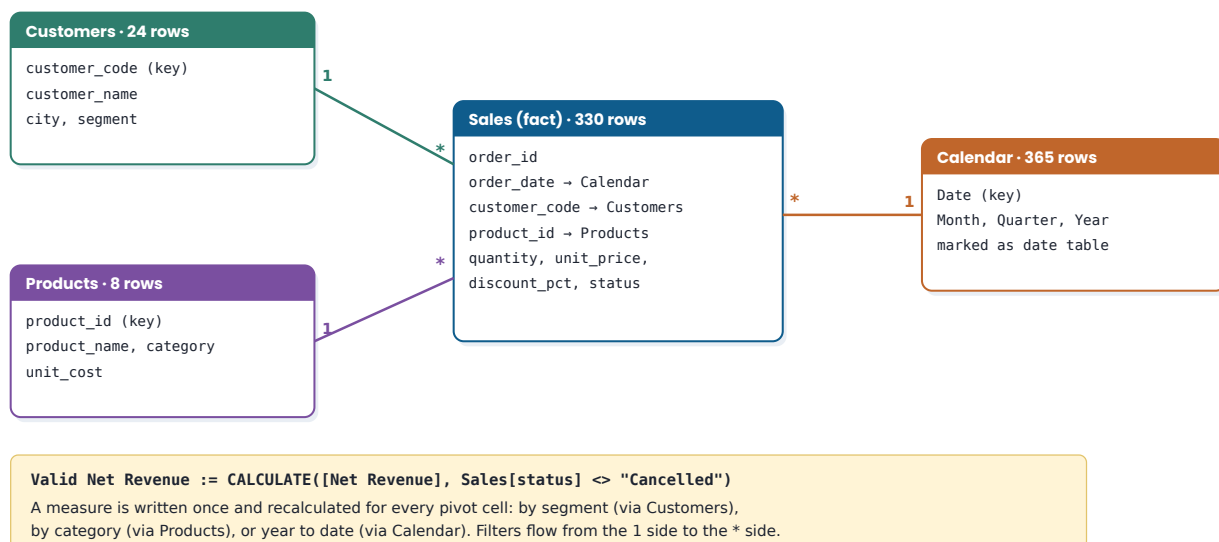


Figure 11.6 – Riverstone's data model: one fact table surrounded by lookup tables. Chapter 12 calls this pattern related tables with keys; Chapter 16 calls it a star schema.

Each relationship is **one-to-many**: one customer, many order lines. Filters flow from the "one" side to the "many" side, so choosing *Wholesale* in Customers filters the sales rows of Wholesale customers.

Measures versus calculated columns

DAX can calculate in two places, and the difference is the most important idea in this section.

- A **calculated column** is computed once for **each row** and stored in the table, like a spreadsheet column. Example: `line_cost = Sales[quantity] * RELATED(Products[unit_cost])`, where `RELATED` fetches the unit cost across the relationship.
- A **measure** is a formula computed **at the moment it's shown**, for whatever the pivot cell represents: all of Retail, Q3 only, or Rahul Mehta in November. Measures are where business definitions live.

Rule of thumb: if you'd filter or group **by** it (a band, a category), make a column. If you'd **add it up, average it, or divide it**, make a measure.

Your first measures

Create a pivot from the model (**Insert → PivotTable → From Data Model**). In the **PivotTable Fields** pane, right-click the sales table → **Add Measure...**, type the name, and enter the formula. (In the Power Pivot window, you can also type measures in the calculation area as `Name := formula`, which is how the companion file `riverstone_measures.dax` lists them.)

```
Net Revenue :=
SUMX ( Sales, Sales[quantity] * Sales[unit_price] * ( 1 - Sales[discount_pct] / 100 ) )
```

`SUMX` goes through the table row by row, calculates the expression for each row, and adds the results: the DAX version of Chapter 10's net revenue column plus `SUM`, without the column.

```
Valid Net Revenue :=
CALCULATE ( [Net Revenue], Sales[status] <> "Cancelled" )
```

CALCULATE evaluates a measure with changed filters: here, "only rows where status isn't Cancelled". It's the most important function in DAX, and Chapter 16 spends a whole section on it. Put **Valid Net Revenue** in Values with **segment** from **Customers** in Rows: Hospitality ₹1,144,038.75, Retail ₹1,488,773.75, Wholesale ₹1,702,658.50, total **₹4,335,471**. ✓ The segment came through the relationship; the sales table doesn't contain it.

```
Gross Margin :=
CALCULATE (
    SUMX ( Sales,
        Sales[quantity] * ( Sales[unit_price] * ( 1 - Sales[discount_pct] / 100 ) - RELATED (
            Products[unit_cost] ) ) ),
    Sales[status] <> "Cancelled" )

Gross Margin % :=
DIVIDE ( [Gross Margin], [Valid Net Revenue] )
```

DIVIDE is a safe division that returns blank instead of an error when the denominator is zero. The results reveal something the revenue view hides:

Segment	Valid net revenue (₹)	Gross margin (₹)	Gross margin %
Hospitality	1,144,038.75	370,338.75	32.4%
Retail	1,488,773.75	443,323.75	29.8%
Wholesale	1,702,658.50	323,558.50	19.0%
Total	4,335,471.00	1,137,221.00	26.2%

Wholesale is Riverstone's biggest segment by revenue and its least profitable: bulk discounts and low-margin Industrial Crates (13.6% gross margin as a category) pull it down. A revenue-only pivot would never show that.

Filter context, in one paragraph

Why does the same measure give ₹1,702,658.50 in the Wholesale row and ₹4,335,471 in the total row? Because every cell of a pivot has a **filter context**: the set of filters that apply to it (this segment, this quarter, whatever the slicers say). A measure is calculated fresh inside each cell's context. **CALCULATE** changes the context on purpose. That idea is the key to all of DAX, and Chapter 16 builds on it.

Time intelligence

With a marked date table, DAX has functions for time comparisons:

```
Revenue YTD :=
TOTALYTD ( [Valid Net Revenue], Calendar[Date] )
```

Put **Calendar[Month]** in Rows (in date order), with **Valid Net Revenue** and **Revenue YTD** in Values. Year to date reaches **₹1,460,879.75** at the end of June, **₹2,581,168.75** at the end of September, and **₹4,335,471** in December. Other common functions: **SAMEPERIODLASTYEAR** for year-on-year comparisons (you'll need more than one year of data, which Chapter 14's full dataset provides) and **DATEADD** for "previous month".

SQL link. A DAX measure in a pivot cell does what Chapter 13's window function **SUM(...)** **OVER** (**ORDER BY** month) does for running totals, and what a filtered **SUM** with **GROUP BY** does for a segment row. Different language, same questions.

11.9 What-if analysis

So far you've asked "what happened?" What-if tools ask "what would it take?" and "what if this changes?"

Goal Seek: work backward to a target

June 2025 was Riverstone's weakest month: ₹186,928 against a ₹300,000 target. Vikram asks: "How many extra Industrial Crates, at ₹1,400 with the usual 8% wholesale discount, would June have needed to hit target?"

Set up a small model on a blank sheet:

	A	B
1	June net revenue so far	186928
2	Extra crates	0
3	Price per crate	1400
4	Discount %	8
5	June net revenue with extra crates	=B1+B2*B3*(1-B4/100)

Excel: Data → Forecast → What-If Analysis → Goal Seek. Set cell: B5 . To value: 300000 . By changing cell: B2 . Click OK.

Goal Seek changes B2 until B5 reaches the target: **87.79** crates. Crates come whole, so the answer is **88**, which gives ₹300,272. You can check the logic without Goal Seek: the gap is ₹113,072, and each crate nets ₹1,288, so $113,072 \div 1,288 = 87.79$. Goal Seek earns its place when the model is too tangled to rearrange by hand.

Google Sheets has no built-in Goal Seek. Google publishes a free **Goal Seek** add-on (**Extensions → Add-ons → Get add-ons**, search "Goal Seek"), or you can solve simple models algebraically as above, or test values in a column and find the first one that reaches the target.

Watch out: Goal Seek overwrites your input. It replaces the value in the changing cell. Note the original, or run it on a copy of the model.

For problems with several changing cells and constraints (a production mix, a delivery schedule), Excel's **Solver** add-in (**File → Options → Add-ins → Excel Add-ins → Solver Add-in**) finds the best combination. It's beyond this chapter, but know it exists.

Data tables: many answers at once

"What would 2026 net revenue be if prices rise 0%, 3%, or 5%, and volumes change by -5% to +10%?" A simple planning model: 2026 revenue = 2025 revenue $\times$ (1 + price change) $\times$ (1 + volume change).

- Put 2025's ₹4,335,471 in B1 , price change in B2 (0), volume change in B3 (0), and in B5 : =B1*(1+B2)*(1+B3) .
- Build a grid: volume changes across row 5 starting in C5 (-5%, 0%, 5%, 10%), price changes down column B starting in B6 (0%, 3%, 5%). The formula cell B5 sits at the grid's top-left corner.
- Select B5:F8 . **Excel:** Data → What-If Analysis → Data Table. **Row input cell:** B3 (volume). **Column input cell:** B2 (price). OK.

Price change ↓ · Volume change →	−5%	0%	+5%	+10%
0%	4,118,697	4,335,471	4,552,245	4,769,018
3%	4,242,258	4,465,535	4,688,812	4,912,089
5%	4,324,632	4,552,245	4,779,857	5,007,469

A 3% price rise with 5% more volume gives about ₹4.69 million. Notice that a 5% price rise with 5% fewer orders (₹4,324,632) still ends slightly *below* 2025.

Excel's data table fills the grid with a special `{=TABLE(B3,B2)}` formula. In **Google Sheets** there's no Data Table command; build the same grid with mixed references (Chapter 10): in C6, `=B$1*(1+$B6)*(1+C$5)`, copied across and down. That version also works in Excel and is easier to audit.

Watch out: data tables recalculate constantly. Large data tables slow Excel down. **Formulas → Calculation Options → Automatic Except for Data Tables** keeps everything else fast; press **F9** to update the tables.

Scenario Manager, briefly

Data → What-If Analysis → Scenario Manager saves named sets of input values ("Cautious", "Plan", "Stretch") for the same cells and produces a summary report comparing their results. It's handy for a quick presentation, but the scenarios are hidden inside the dialog. Many analysts prefer a visible **scenario table**: one column per scenario, one row per input, and a drop-down (data validation) that picks which column feeds the model. That approach works in both apps.

11.10 Google Sheets power features

Google Sheets has several features with no direct Excel equivalent. They're the reason many analysts who live in Google Workspace can do serious work without Power Query.

QUERY: SQL-like formulas

`QUERY(data, query, [headers])` runs a query written in the **Google Visualization API Query Language**, which looks a lot like the SQL you'll learn in Chapter 12. It refers to columns by their **letters** within the data range.

"Revenue by segment, excluding cancelled orders, largest first."

```
=QUERY(Sales!A1:S331,
  "select N, sum(J)
  where H <> 'Cancelled'
  group by N
  order by sum(J) desc
  label sum(J) 'net_revenue'", 1)
```

The result spills into a small table: Wholesale 1702658.5, Retail 1488773.75, Hospitality 1144038.75. The final 1 says the range has one header row.

How it works:

- `select N, sum(J)` : column N (segment) and the sum of column J (net revenue).

- where `H <> 'Cancelled'` : text values go in **single** quotes inside the query string, which itself is in double quotes.
- `group by N` , `order by sum(J) desc` , and `label` to rename a column: the same clauses and order as SQL.
- **pivot** spreads a column across the top: `select N, sum(J) where H <> 'Cancelled' group by N pivot L` rebuilds Figure 11.3's segment-by-quarter grid in one formula.
- Dates use a keyword: `where K = date '2025-11-01'` . Build the string from a cell with `"where K = date '&TEXT(B1,"yyyy-mm-dd")&' "` .

Watch out: mixed types in a column. `QUERY` decides each column's type from the majority of its values and treats the minority as empty. A code column with some numbers and some text (`14` and `0014`) will lose one kind silently. Clean types first (Chapter 10, section 10.4).

SQL link. `QUERY` is excellent practice for Chapter 12, but it isn't SQL: no joins, no subqueries, and column letters instead of names. When your `QUERY` formulas start needing joins, the data belongs in a database.

ARRAYFORMULA: one formula for a whole column

In Google Sheets, `ARRAYFORMULA` makes a formula work on whole ranges, so one cell fills a column:

```
=ARRAYFORMULA(IF(E2:E="", "", E2:E * F2:F * (1 - G2:G/100)))
```

Put it in `J2` and every row with a quantity gets its net revenue, including rows added later. The `IF(E2:E="", "", ...)` part leaves empty rows blank instead of showing zeros down the rest of the sheet. Many Sheets functions (including `FILTER` , `SORT` , `UNIQUE` , and `QUERY`) already work on arrays and don't need it. In Excel, ordinary formulas spill in the same way (section 11.6), so `ARRAYFORMULA` isn't needed.

Paste this one formula into the Chapter 10 practice file in Sheets, and you have the net revenue column without filling down. It's the closest Sheets comes to a Power Query calculated column.

IMPORTRANGE: pull data from another spreadsheet

```
=IMPORTRANGE("https://docs.google.com/spreadsheets/d/...", "Sales!A1:S331")
```

The first argument is the source spreadsheet's URL (or its ID), the second is the range as text. The first time, the cell shows `#REF!` with **Allow access**; click it to connect the two files. After that, the data stays live: when the source changes, the import updates.

Combine it with `QUERY` to summarize another team's file without copying it. Inside `QUERY` , imported columns are referred to as `Col1` , `Col2` ... instead of letters:

```
=QUERY(IMPORTRANGE("https://docs.google.com/spreadsheets/d/...", "Sales!A1:S331"),
  "select Col14, sum(Col10) where Col8 <> 'Cancelled' group by Col14", 1)
```

Watch out: IMPORTRANGE and permissions. Once access is allowed, **anyone who can edit your file** can import any range from the source file. Don't connect a file with salaries or customer contact details to a widely shared report. Very large imports can also be slow or fail; import only the columns you need.

For stacking several ranges (twelve monthly tabs, say), both apps have `VSTACK : =VSTACK(Jan!A2:I16, Feb!A2:I21, ...)`. In Sheets, the older `{Jan!A2:I16; Feb!A2:I21}` array syntax does the same.

Sheets pivot tables and slicers

You built Sheets pivot tables in section 11.5. Two Sheets-specific points: they **update automatically** without a refresh, and **slicers** (**Data → Add a slicer**) filter every chart and pivot on the same data range, which makes a simple interactive dashboard on one sheet. For a proper dashboard, Looker Studio (Chapter 16) reads Google Sheets directly.

Connecting Sheets to a data warehouse

When data outgrows a spreadsheet, it usually moves to a **data warehouse** (Chapter 49 explains them).

Connected Sheets lets Google Sheets work with Google BigQuery tables that are far too big for a sheet: **Data → Data connectors → Connect to BigQuery**. Pivot tables, charts, and formulas on a connected sheet run as queries in BigQuery, and you can pull an **extract** of up to 500,000 rows (subject to cell limits) into the sheet. It's available in Google Workspace editions that include it, and you need access to BigQuery through your company.

Excel's equivalent is Power Query's database connectors: **Data → Get Data → From Database** (SQL Server and others, depending on your edition), or **From Other Sources → From ODBC** for databases such as PostgreSQL and MySQL once an ODBC driver is installed. Chapter 12's databases can feed a pivot table this way, with the query running in the database and only the result coming into Excel.

11.11 What Excel does that Sheets can't (and the other way round)

Need	Excel	Google Sheets	Sheets alternative
Combine and clean files with recorded steps	Power Query	No	<code>IMPORTRANGE</code> + <code>VSTACK</code> + <code>QUERY</code> ; Apps Script (Chapter 19); or clean in SQL or Python first
Several related tables with measures	Power Pivot, DAX (Windows)	No	<code>QUERY</code> over pre-joined data; Looker Studio or Connected Sheets; Power BI (Chapter 16)
Very large data inside the file	Data Model (millions of rows)	10 million cells	Connected Sheets with BigQuery
What-if tools	Goal Seek, Data Table, Scenario Manager, Solver	Goal Seek add-on	Mixed-reference grids; scenario tables
Formula summaries	<code>GROUPBY</code> , <code>PIVOTBY</code> (Microsoft 365)	No	<code>QUERY</code> with <code>group by</code> and <code>pivot</code>
Refer to a spill range	<code>F2#</code>	No	Open-ended ranges, <code>ARRAYFORMULA</code>
Macros	VBA, Office Scripts (Chapter 19)	Apps Script (Chapter 19)	—

And the reverse:

Need	Google Sheets	Excel	Excel alternative
SQL-like formulas	<code>QUERY</code>	No	Power Query; <code>FILTER</code> + <code>GROUPBY</code> ; pivot tables

Need	Google Sheets	Excel	Excel alternative
Live link to another file with a formula	IMPORTRANGE	External links (fragile when files move)	Power Query From Workbook or From SharePoint Folder
Web data in a formula	IMPORTHTML , IMPORTXML , IMPORTDATA , GOOGLEFINANCE	No direct equivalent	Power Query From Web ; the Stocks data type
Real-time co-editing everywhere	Always	Cloud files with AutoSave	Store on OneDrive or SharePoint
Forms feeding a sheet	Google Forms	Microsoft Forms (work and school accounts)	—

Simplification note. Both products change every few months. Microsoft has added REGEX functions, Python in Excel, and Copilot features; Google has added tables, AI features, and new connectors. Treat this table as a guide to the *kinds* of differences and check the current help pages before you choose a tool for a team.

11.12 Spreadsheet hygiene and auditing

A workbook that other people rely on needs the same care as code. These habits prevent most errors and make the rest findable.

A structure that survives handover

1. **A README sheet first.** What the workbook is for, who owns it, where the data comes from, how to refresh it, the business rules ("net revenue excludes cancelled orders"), and a change log with dates.
2. **Inputs, calculations, outputs.** Raw data and assumptions on their own sheets; calculations in between; the report on its own sheet. Color input cells consistently (a light yellow fill is common) so readers know what they may change.
3. **No hard-coded numbers inside formulas.** `=B5*1.18` hides a tax rate; put `18%` in a labeled input cell or a named range (`tax_rate`) and refer to it.
4. **One formula per column.** Every row of a calculated column should have the same formula. Excel flags a formula that differs from its neighbors with a green triangle (**Inconsistent Formula**); take those warnings seriously.
5. **Tables and queries, not fixed ranges.** They grow with the data (sections 10.10 and 11.7).
6. **Avoid fragile functions** where you can: `INDIRECT` and `OFFSET` build references from text or positions, break silently when sheets are renamed, and recalculate constantly.
7. **Check cells everywhere.** Every summary should have a visible check that compares its total with an independent total, showing 0 or "OK". Chapter 10's tracker did this; section 11.7's refresh test is another.

Auditing tools

To do this	Excel	Google Sheets
See which cells feed a formula	Formulas → Trace Precedents / Trace Dependents	Click into the formula; ranges are outlined
Step through a formula	Formulas → Evaluate Formula	—

To do this	Excel	Google Sheets
Show all formulas	Ctrl+`	Ctrl+`
Find all formulas, constants, or errors	Home → Find & Select → Go To Special	Ctrl+` , then Find
List queries and connections	Data → Queries & Connections	—
Find links to other workbooks	Data → Edit Links (Workbook Links in newer versions)	Search for <code>IMPORTRANGE</code>
Workbook size and contents	Review → Workbook Statistics	—
Compare two versions of a file	Inquire → Compare Files (Inquire add-in, enterprise editions)	File → Version history
Find hidden sheets	Right-click a sheet tab → Unhide	View → Hidden sheets

An audit checklist for an inherited workbook

1. **Where does the data come from, and when was it last updated?** Look for pasted data, queries, and links.
2. **Does the headline total reconcile** to an independent source (the database, finance, Chapter 10's tracker)?
3. **Are the rules the same everywhere?** Compare the formulas across months or sections, not only the results.
4. **Are any numbers typed in** where a formula should be? **Go To Special → Constants** in a summary area finds them.
5. **Do the ranges cover all the data?** Check the last row each formula uses against the last row of data.
6. **Are there duplicates or missing periods** in the data?
7. **Are there hidden sheets, rows, or columns** with numbers that feed the report?
8. **Write down what you found** and what you changed, in the README.

The story after this section runs exactly this checklist on `month_end_pack_2025_messy.xlsx`.

11.13 When to leave the spreadsheet

Spreadsheets are superb for exploring, modeling, and small-to-medium reporting. They're the wrong home for some jobs. Watch for these signals:

Signal	What it looks like	Better home
Size	Hundreds of thousands of rows; slow recalculation; files over tens of MB	A database (Chapter 12), Parquet files, the Data Model
Many sources joined every time	Lookups across five exports each month	SQL joins (Chapter 12), Power Query at the least
Many people changing the data	Conflicting edits; "who changed this?"	A database with access controls; a proper app
It must run without a person	"Refresh this every morning at 7"	Scheduled scripts and pipelines (Chapters 18 and 20); the Power BI service (Chapter 16)
An error would be costly	Invoices, payroll, regulatory reports	Tested code with version control (Chapters 18 and 26)
Logic you can't see	Nested formulas no one can explain	Code with comments, or Power Query steps

Signal	What it looks like	Better home
Interactive dashboards for many users	Pivot-and-slicer sheets emailed around	A BI tool (Chapter 16)

None of this means abandoning spreadsheets. The usual pattern in a mature team is: data lives in a database or warehouse, SQL or a pipeline prepares it, and Excel or Sheets connects to the prepared data for the last mile of analysis and presentation.

11.14 Competition-style Excel: a guided championship case

Everything so far has been about building workbooks that last. Competitive Excel tests the other half of mastery: turning an unfamiliar problem into a working model **fast**, under a clock, with no one to ask. It's the most demanding practice you can do with a spreadsheet, and the habits it builds (read the whole problem first, parse data into clean columns, keep inputs in cells, check small cases by hand) are exactly the habits of a strong analyst.

What competitive Excel looks like

The best-known event is the **Microsoft Excel World Championship (MEWC)**, organized by the Financial Modeling World Cup with Microsoft as a sponsor. At the time of writing, its format works like this:

- Each round is one **case**: a fictional scenario (a game, a puzzle, a simulation) described in a workbook, with a time limit of about **30 minutes**. Finals run longer, and players with the lowest scores are eliminated at intervals.
- Questions are grouped into **levels** that get harder and are worth more points, plus **bonus** questions. A full case is worth around 1,000 points.
- Answers are **numbers or short text** typed into a platform and graded automatically, so any method that gets the right answer is allowed: formulas, helper columns, pivot tables, Power Query, VBA.
- Cases don't need finance or domain knowledge. They test functions, data handling, and logical thinking, and they're written so older Excel versions can solve them, although newer functions are often faster.

The related **Financial Modeling World Cup** runs longer finance-flavored cases, and there's a collegiate championship for students. Rules change between seasons, so read the current rules on the organizer's site before you enter.

Tool note. You don't need to compete to benefit. Past cases and walkthrough videos are published by the organizers and by top players, and working through them, with a timer, is some of the best Excel practice available. Try them after this section.

How strong players attack a case

1. **Read every level before touching a formula (3 minutes).** Later levels usually reuse the model from earlier ones with a twist. If Level 5 needs a search over many settings, build Level 3's model with its settings in **input cells**, not typed into formulas.
2. **Parse once, into clean columns.** Case data often arrives as codes or text. Turn it into typed columns (dates, numbers, text) on a helper sheet, then never touch the raw text again.

3. **Build one row per unit of time or event.** Simulations become a table with one row per day, turn, or move, where each row's formulas read the row above. That's the running-balance pattern from section 11.2, stretched into a model.
4. **Answer the early levels immediately.** Points are points. Type Level 1's answers as soon as you have them.
5. **Check tiny cases by hand.** Before trusting a 365-row model, trace the first few rows (or one week) on paper.
6. **Search with a data table.** "Which combination is cheapest?" is a two-input data table (section 11.9) over the model's input cells, followed by `MIN` and a two-way lookup.
7. **Keep a scratch area.** Label intermediate answers so you can find them again at minute 28.

The case: The Riverstone Stockroom

Open `companion/ch11/challenge/riverstone_stockroom_case.xlsx`. **Give yourself 30 minutes** and try it before reading the walkthrough. Its **Case** sheet gives the brief; the **Log** sheet holds Riverstone's 330 order lines for 2025, each squeezed into one text code; **Parameters** holds the settings; **Answers** has cells for your answers.

Riverstone's warehouse wants to know whether a simple weekly reorder rule would have kept Storage Box 10L (product 101) in stock in 2025.

Each log line looks like `02.01.2025|S010001|C0002|P108x10@290-0%|DEL`: date, order, customer, product × quantity @ price – discount %, status (DEL delivered, SHP shipped, PND pending, CAN cancelled).

Rules (product 101 only; cancelled lines never ship):

1. The model runs day by day from 1 January to 31 December 2025. Demand on a day is the units of product 101 on non-cancelled lines dated that day.
2. Opening stock on 1 January is 300 units. Closing stock = opening stock + units received that day – demand. Stock may go negative (a backorder).
3. Every **Monday**, if the previous day's closing stock is less than or equal to the **reorder point (ROP)**, a purchase order (PO) for **Q** units is placed.
4. A PO placed on day d arrives at the start of day $d + L$ (L = lead time in days).
5. Costs: holding ₹1 per unit per day of positive closing stock; shortage ₹20 per unit per day of negative closing stock; ₹2,000 per PO.

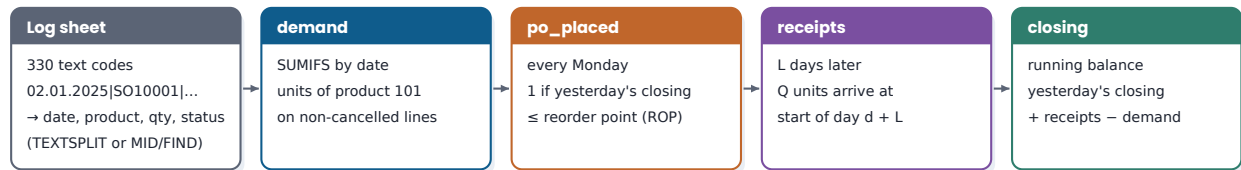
Level 1 (100 points): (a) total units on non-cancelled lines, all products; (b) the product ID with the most units; (c) for product 101, the customer code that bought the most units. **Level 2 (150):** with no reordering, the first date closing stock goes below zero, and the closing stock on 31 December. **Level 3 (200):** with ROP 100, Q 250, L 7: how many POs, the lowest closing stock, and its first date. **Level 4 (250):** same settings: how many days is closing stock negative? Keeping Q = 250, the smallest ROP (a multiple of 50) with no negative day. **Level 5 (300):** L = 7; ROP can be 0, 50, ... 400 and Q can be 50, 100, ... 800. The lowest-cost combination and its cost. **Bonus (50):** the same search with L = 14; the lowest cost.

Stop here and try it. The walkthrough follows.

Minutes 0–3: read, and plan the model

Levels 2 to 5 all need the same thing: a table with one row per day, and ROP, Q, and L in input cells. Level 1 needs the log parsed. Figure 11.7 shows the plan.

The Riverstone Stockroom: one row per day, 1 Jan to 31 Dec 2025 (Storage Box 10L)



closing feeds next Monday's decision

date	wkday	demand	receipts	po	closing
2025-02-09	7	0	0	0	95
2025-02-10	1	0	0	1	95
2025-02-16	7	25	0	0	70
2025-02-17	1	30	250	1	290

ROP 100, Q 250, lead time 7 days:

- 8 purchase orders; lowest closing –45 on 3 Oct
 - 7 days below zero; year-end closing 215
 - total cost ₹118,210 (holding + shortage + orders)
- 17 Feb: 250 units arrive, yet another order is placed, because the rule reads Sunday's stock, not stock on order.

Figure 11.7 – The Stockroom model: parse the log once, then one row per day where each column reads the columns and rows before it.

Minutes 3–8: parse the log (and answer Level 1)

On the Log sheet, add typed columns next to the code in column A. The codes have fixed positions for the date, order, customer, and product, and variable lengths after that, so combine **MID** with **FIND**:

```

B2 order_date      =DATE(MID(A2,7,4), MID(A2,4,2), LEFT(A2,2))
C2 order_id        =VALUE(MID(A2,14,5))
D2 customer_code   =MID(A2,21,4)
E2 product_id      =VALUE(MID(A2,27,3))
F2 quantity        =VALUE(MID(A2,31,FIND("@",A2)-31))
I2 status          =RIGHT(A2,3)
  
```

How it works: in 02.01.2025|S010001|C0002|P108x10@290-0%|DEL, the year starts at character 7, the order number at 14, the customer code at 21, and the product at 27. The quantity starts at 31 and ends immediately before @, so its length is **FIND("@",A2)-31**. Fill all six down to row 331.

In Microsoft 365, one formula can split the whole code into pieces:

```
=TEXTSPLIT(A2, {"|","x","@", "-","%"} )
```

It returns 02.01.2025, S010001, C0002, P108, 10, 290, 0, an empty piece (where % and | sit side by side), and DEL. You'd then convert the pieces with **VALUE** and **SUBSTITUTE**. Use whichever you can write fastest and check.

Level 1 is now three quick formulas:

```

=SUMIFS(F2:F331, I2:I331, "<>CAN")           → 9475
=SUMIFS(F2:F331, E2:E331, 101, I2:I331, "<>CAN") → 2085
=SUMIFS(F2:F331, E2:E331, 101, D2:D331, "0012", I2:I331, "<>CAN") → 250
  
```

(a) **9,475** units. (b) A small **UNIQUE** list of product IDs with a **SUMIFS** beside it (or a pivot) shows product **101** leads with 2,085 units, ahead of the Stackable Bin's 1,920. (c) The same pattern by customer code for product 101 gives **0012** (Fresh Bowl Kitchens) with **250** units, ahead of two customers tied on 235.

Watch out: the cancelled lines are in the log. Without "<>CAN", total units come to 9,680. Competition cases hide rules like this in one line of the brief, and business data hides them in one column.

Minutes 8–15: build the daily model (and answer Level 2)

Put the settings on a Parameters sheet: opening stock in B1 (300), product in B2 (101), ROP in B3, Q in B4, L in B5, and the three costs in B6:B8. Then, on a Model sheet with headers in row 1:

```
A2 date      =DATE(2025,1,1)
A3           =A2+1                      (fill down to row 366)
B2 weekday   =WEEKDAY(A2,2)              (Monday = 1)
C2 demand    =SUMIFS(Log!$F$2:$F$331,
                    Log!$B$2:$B$331, A2,
                    Log!$E$2:$E$331, Parameters!$B$2,
                    Log!$I$2:$I$331, "<>CAN")
E2 po_placed 0
E3           =IF(AND(B3=1, F2<=Parameters!$B$3), 1, 0)
D2 receipts  =IF(ROW()-Parameters!$B$5>=2,
                INDEX($E$1:$E$366, ROW()-Parameters!$B$5)
                * Parameters!$B$4, 0)
F2 closing   =Parameters!$B$1+D2-C2
F3           =F2+D3-C3
```

How it works:

- demand is a SUMIFS by date (section 11.2), one per day.
- po_placed on row 3 reads **yesterday's** closing (F2), exactly as rule 3 says. It's 1 only on Mondays when that stock is at or below the ROP.
- receipts looks **L rows up** the po_placed column: if a PO was placed L days ago, Q units arrive today. INDEX(\$E\$1:\$E\$366, ROW()-L) is the lookup; the IF stops it from looking above the first row.
- closing is the running balance.

For Level 2, set ROP to a value that never triggers, or temporarily set Q to 0, and read the answers:

```
=INDEX(A2:A366, MATCH(TRUE, INDEX(F2:F366<0,0), 0)) → 2025-02-28
=F366 → -1785
```

The first formula finds the first TRUE in the test "closing is below zero"; the inner INDEX(...,0) makes it work without **Ctrl+Shift+Enter** in older Excel. Stock first goes negative on **28 February 2025** (-10), and without reordering the year ends at **-1,785**, which is 300 minus the year's 2,085 units of demand. That last check (opening stock minus total demand) proves the model's demand column is complete.

Try it. Trace the first week by hand before trusting the model: 300 units on 1 January, and the first demand for product 101 arrives on 12 January (25 units, order 10003). If your closing stock changes before 12 January, a formula is reading the wrong product or date.

Minutes 15–20: Levels 3 and 4

Set ROP = 100, Q = 250, L = 7. Then:

POs placed	=SUM(Model!E2:E366)	→ 8
lowest closing	=MIN(Model!F2:F366)	→ -45
date of lowest	=INDEX(Model!A2:A366, MATCH(MIN(Model!F2:F366), Model!F2:F366, 0))	→ 2025-10-03
negative days	=COUNTIF(Model!F2:F366, "<0")	→ 7

Level 3: 8 POs; the lowest closing stock is **-45**, first reached on **3 October 2025**. **Level 4:** stock is negative on 7 days. For the smallest safe ROP, change ROP to 150, 200, ... and watch the negative-day count: at **150** it drops to 0 (lowest closing 55). A one-input data table over ROP = 0, 50, ... 400 shows the same answer in one step.

Figure 11.8 shows why ROP 100 isn't safe: stock sits above the reorder point on Sunday night, the following week's orders drain it, and the next PO is still a week away from arriving.

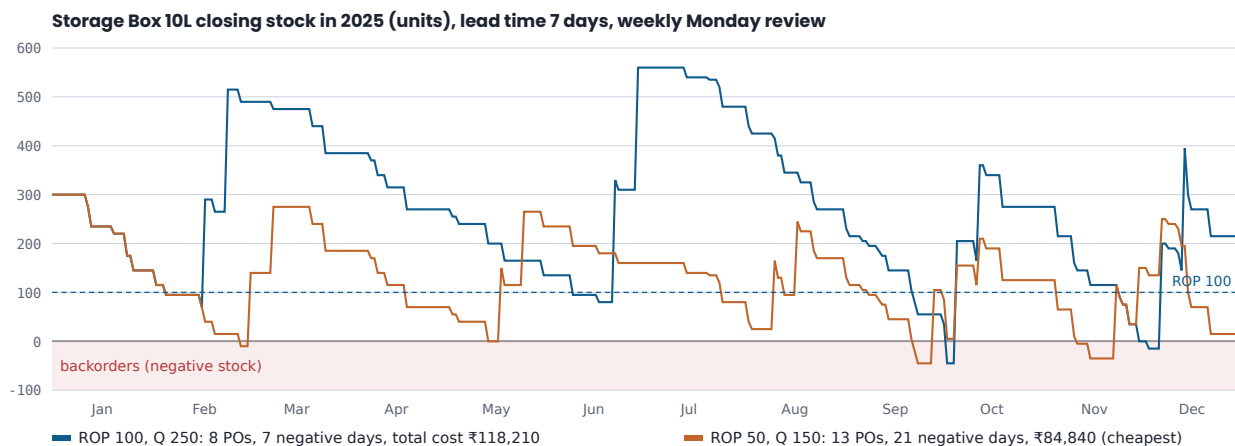


Figure 11.8 — Closing stock under the Level 3 policy (ROP 100, Q 250) and the Level 5 winner (ROP 50, Q 150). The cheapest policy runs short more often, because a day of backorder is cheaper than a week of carrying 250 extra boxes.

Minutes 20–28: Level 5 with a data table

Add three cost cells beside the model and a total:

holding	=SUMIF(Model!F2:F366, ">0") * Parameters!B6
shortage	=-SUMIF(Model!F2:F366, "<0") * Parameters!B7
ordering	=SUM(Model!E2:E366) * Parameters!B8
total	=holding + shortage + ordering

At ROP 100, Q 250 the total is **₹118,210** (holding ₹98,310, shortage ₹3,900, orders ₹16,000).

Now the search. Build a grid with the total cost cell in its top-left corner, ROP values 0 to 400 down the first column, and Q values 50 to 800 across the first row: 9 rows × 16 columns = 144 combinations. Put the grid on the Parameters sheet, select it, and use **Data → What-If Analysis → Data Table**, with **Row input cell** = the Q cell (B4) and **Column input cell** = the ROP cell (B3). Excel reruns the whole 365-day model for every cell of the grid.

Watch out: data table inputs must be on the same sheet. Excel's Data Table dialog only accepts input cells on the sheet that holds the table, which is why the grid goes on the Parameters sheet. Google Sheets has no data table: loop the settings with a small Apps Script (Chapter 19) or copy the model once per setting.

Then find the minimum and where it is:

```
lowest cost  =MIN(grid)                                → 84840
ROP          =INDEX(rop_list, SUMPRODUCT((grid=MIN(grid))*ROW(grid))-MIN(ROW(grid))+1)
Q            =INDEX(q_list, SUMPRODUCT((grid=MIN(grid))*COLUMN(grid))-MIN(COLUMN(grid))+1)
```

(SUMPRODUCT returns the row and column of the single cell that equals the minimum; if two combinations tied, you'd use a helper grid instead.) In Microsoft 365, `=LET(c, grid, m, MIN(c), TOCOL(IF(c=m, rop_list&" / "&q_list, 1/0), 2))` lists every winning combination.

Level 5: ROP 50, Q 150, total cost **₹84,840**, with 13 POs and 21 negative days. The next-best combinations are ROP 100/Q 150 (₹95,300) and ROP 100/Q 100 (₹96,165).

Bonus: set $L = 14$ and let the data table recalculate. The lowest cost rises to **₹112,125**, at ROP 50, Q 200. A longer lead time costs Riverstone about ₹27,000 a year for this one product.

Minutes 28–30: sanity checks before submitting

- Opening stock + all receipts – all demand = the 31 December closing stock (with ROP 100/Q 250: $300 + 8 \times 250 - 2,085 = 215$ ✓).
- The number of POs $\times$ Q equals total receipts.
- The best cost is lower than the Level 3 policy's cost, and the winning settings are inside the search range, not on its edge (if the best Q had been 800, the search range would be too narrow).

What the case teaches beyond the competition

The winning policy is cheapest *under the costs in the brief*. A shortage cost of ₹20 a day per box ignores lost customers and a sales head's phone calls. A real recommendation would test several shortage costs, show the trade-off between cost and backorder days (Figure 11.8 does exactly that), and let the business choose. Competitions reward the number; analysis work rewards the number plus its assumptions (Chapters 22 and 24).

The model itself is also a template: daily inventory, cash balances, subscription counts, and loyalty points are all "yesterday's value + today's in – today's out" with a decision rule. You'll meet the same pattern in SQL (Chapter 13's running totals) and Python (Chapter 18).

Your turn: a timed practice case, Riverstone Rewards (45 minutes)

Open `challenge/riverstone_rewards_case.xlsx`. It has a **Rules** sheet, a **Questions** sheet, and a **Lines** sheet with the 330 order lines of 2025 (customer names, not codes, and cancelled lines included). Set a timer for 45 minutes. No walkthrough this time: plan the model yourself. The answers are at the end of the chapter.

Rules

1. Cancelled orders earn nothing and don't count for any rule.
2. **Order value** = the sum of `net_revenue` over the order's lines.
3. **Base points** = order value $\div$ 100, rounded down to a whole number.
4. **Tier** at the time of an order depends on the customer's total order value from their **earlier** orders in 2025 (orders with a smaller `order_id`): below ₹100,000 is **Standard** ($\times 1$); ₹100,000 or more is **Silver** ($\times 1.25$); ₹250,000 or more is **Gold** ($\times 1.5$). **Tier points** = base points $\times$ multiplier, rounded down.
5. **Streak bonus**: +200 points on an order if the same customer placed at least one order in the previous calendar month.
6. **Basket bonus**: +100 points on an order with 3 or more lines.
7. **Order points** = tier points + streak bonus + basket bonus.

8. **Expiry** on 31 December 2025: points from orders dated before 1 July 2025 expire, unless the customer ordered on or after 1 October 2025.
9. **Balance** = all order points – expired points.

Questions

- **L1:** How many orders earn points, and how many of them are worth ₹25,000 or more?
- **L2:** Total base points? Which customer has the most base points?
- **L3:** How many orders were placed at Gold tier, and how many at Silver?
- **L4:** Total tier points (after multipliers, before bonuses)?
- **L5:** How many orders earn the streak bonus? Total of tier points plus streak bonuses?
- **L6:** Total order points across all orders?
- **L7:** How many customers lose points to expiry, and how many points expire?
- **L8:** The highest balance (customer and points), and how many customers finish with 4,000 points or more?
- **B1:** Which customer's running points (before expiry) first reach 5,000, and on which `order_id`?
- **B2:** Which month earned the most order points, and how many?
- **B3:** How many customers ordered in every month of 2025?
- **B4:** Metro Mart's rank by balance (1 = highest)?

Hints, if you're stuck after 10 minutes. Build an **Orders** sheet with one row per valid order (`UNIQUE(FILTER(...))` , a pivot pasted as values, or Power Query **Group By**). "Earlier orders" is an **expanding range** (`SUMIFS($D$2:D2, $C$2:C2, C2) - D2`). "Previous month" is `COUNTIFS` with `EDATE(month, -1)` . [challenge/riverstone_rewards_solution.xlsx](#) is a complete solution that uses only classic functions, so it works in every Excel version and in Google Sheets.

Common mistakes and how to spot them

Mistake	Symptom	Fix
<code>VLOOKUP</code> without <code>FALSE</code>	Plausible but wrong names or prices	Use <code>XLOOKUP</code> (exact by default) or <code>VLOOKUP(..., FALSE)</code>
<code>VLOOKUP</code> column number after inserting a column	A lookup suddenly returns a different field	<code>XLOOKUP</code> or <code>INDEX / MATCH</code> , which point at the return column
Approximate match on an unsorted band table	Wrong band for some values	Sort thresholds ascending; test values slightly below and above each threshold
Counting <code>order_id</code> in a pivot	"105 Wholesale orders" when there were 55	Distinct Count (Data Model) or <code>COUNTUNIQUE</code> in Sheets
Forgetting to refresh an Excel pivot	Pivot total doesn't match the data	Refresh All ; refresh on open; check cells
Pivot built on a fixed range	New rows missing from the pivot	Build on a table; Change Data Source
Typing inside a spill range	<code>#SPILL!</code> (Excel) or <code>#REF!</code> (Sheets)	Clear the obstruction; keep spill areas empty
<code>FILTER</code> without an "if empty" value	<code>#CALC!</code> when nothing matches	Add the third argument: "no rows"

Mistake	Symptom	Fix
Accepting Power Query's automatic Changed Type step	Codes lose their zeros; dates misread	Delete it; set types yourself, dates Using Locale
Merging on columns of different types	"Matches 0 of 326 rows"	Same type on both sides before merging
A new export with a changed layout	Refresh fails or columns shift	Check row counts and totals after each refresh; agree a fixed export layout
Calculated columns for things that should be measures	Percentages that add up instead of recalculating	Ratios and totals as measures; columns only for per-row attributes
Summing a percentage in a pivot	Total row shows 300%	A measure such as <code>DIVIDE([Gross Margin], [Valid Net Revenue])</code>
Goal Seek overwriting an input	The model's original assumption is lost	Note the value or work on a copy
QUERY on a column with mixed types	Some values vanish from results	Clean types before querying
Sharing a file that uses IMPORTRANGE widely	Editors can pull data from the source file	Limit who can edit; import only needed columns
Different rules in different months	Some months include cancelled orders	One rule, applied once (query, measure, or single formula)
Numbers typed into a summary	A total that doesn't change when data changes	Go To Special → Constants ; replace with formulas

In the real world: the month-end pack that was almost right

In the first week of January 2026, Anita Rao, Riverstone's Sales Head, was preparing the sales section of the quarterly board review. Vikram Singh sent her the Summary sheet from `month_end_pack_2025_messy.xlsx`: 2025 net revenue ₹4,270,875, 100.7% of target. Two days later, Meera Iyer's Chapter 10 tracker, built from the same ERP data, said ₹4,335,471, 102.3%.

The gap was ₹64,596. Anita asked a simple question: "Which one goes to the board?"

Meera ran the audit checklist from section 11.12.

1. Where does the data come from? Twelve month sheets pasted from the ERP's monthly exports, and a Summary sheet with one formula per month. No queries, no links.

2. Does the headline reconcile? No: ₹64,596 short of the tracker and of the database query from Chapter 13. So she compared month by month:

Month	Summary sheet	Tracker	Difference
Apr	235,081.50	210,281.50	+24,800.00
Oct	545,047.75	681,070.75	−136,023.00
Nov	633,480.00	633,408.00	+72.00
Dec	486,378.50	439,823.50	+46,555.00
Year	4,270,875.00	4,335,471.00	−64,596.00

The other eight months matched exactly. Four months, four different problems.

3. Are the rules the same everywhere? She pressed **Ctrl+ **** on the Summary sheet and read the formulas. Most months used **SUMIFS(...,<>Cancelled)**. **March and April used **SUM'**, which includes cancelled orders. April had one cancelled order, 10034, worth ₹24,800: that was April's difference. March had no cancelled orders, so its number was right by *luck*, with the wrong formula. The next time a March order was cancelled, the report would be wrong again. She noted it as a defect even though it cost nothing today.

4. Are any numbers typed in? Home → Find & Select → Go To Special → Constants on the Summary's revenue column selected one cell: **November**, a typed **633,480**, with a note "typed from the flash email". The real figure was ₹633,408. Two digits had been swapped. A ₹72 difference is harmless on its own, but it proved that the report wasn't connected to its data.

5. Do the ranges cover the data? October's formula was **SUMIFS(Oct!J2:J30, Oct!H2:H30, "<>Cancelled")**. The October sheet had **38 lines**, in rows 2 to 39. Someone had built the formula when October's paste was shorter, and the last **9 lines**, worth **₹136,023**, were never counted. October showed 104.8% of target when it had really reached 131.0%, Riverstone's best month.

6. Duplicates? December's sheet had 40 lines; the ERP export had 35. **COUNTIFS** on order ID and product showed the **last five lines pasted twice**, adding ₹46,555. December showed 128.0% of target instead of 115.7%.

Four errors: a wrong rule in two months (one of them hidden), a typed number, a short range, and a double paste. They partly cancelled each other out, which is why the Summary looked believable.

7. Fix the process, not the cells. Correcting four cells would have made 2025 right and left 2026 exposed to the same four errors. Instead, Meera rebuilt the pack as sections 11.5 and 11.7 describe:

- A **Power Query** on the monthly export folder, with types set by hand and the cancelled filter applied once.
- A **pivot table** on the query's table (month in Rows, revenue and target in Values), with a slicer for segment.
- A **check cell**: pivot grand total minus **SUM** of the query's net revenue column, which must be 0, and a row count that must equal the sum of the files' lines.
- A **README** sheet stating the rule, the source folder, and "Data → Refresh All; don't paste".

She tested the refresh by removing December's file (total ₹3,895,647.50) and putting it back (₹4,335,471).

8. Answer the question. Her note to Anita:

Use ₹4,335,471 (102.3% of target). The ₹4,270,875 version had four errors: October was missing ₹136,023 of orders, December had ₹46,555 counted twice, April included a ₹24,800 cancelled order, and November was typed in ₹72 high. The corrected October (131% of target) is our best month, not an average one. The pack is now built from the export folder and refreshes in one click, with checks that flag any difference.

The board saw the right number. In February, Vikram refreshed the pack himself.

What made the difference:

- Meera **reconciled month by month**, which turned "₹64,596 is missing" into four specific rows.
- She **read the formulas, not only the results**, and caught a bug that happened to cost nothing this year.
- She **looked for typed numbers and short ranges**, the two errors copy-paste workbooks produce most often.
- She **replaced the process** that produced the errors, and **added checks** so the next error announces itself.

Tools

- **Microsoft Excel for Windows**, Microsoft 365: everything in this chapter. Power Pivot and the Data Model require Windows. `XL00KUP` , dynamic arrays, and `LET` need Microsoft 365, Excel 2021, or later; `GROUPBY` and `PIVOTBY` need a current Microsoft 365 version.
 - **Excel for Mac** (Microsoft 365): lookups, pivot tables, dynamic arrays, what-if tools, and Power Query with fewer sources. No Power Pivot.
 - **Google Sheets**: lookups, pivot tables, slicers, dynamic arrays, `QUERY` , `ARRAYFORMULA` , `IMPORTRANGE` ; the Google **Goal Seek** add-on; **Connected Sheets** in Workspace editions that include BigQuery.
 - **Free alternative**: LibreOffice Calc has pivot tables (called DataPilot or Pivot Tables), `INDEX / MATCH` , Goal Seek, and Multiple Operations (a data table), but no Power Query or Power Pivot.
 - **Companion files** (`companion/ch11/` , Appendix E):
 - `ch11_practice.xlsx` : Sales (330 lines, 19 columns), Customers, Products, Targets, TargetsWide, and RebateBands.
 - `monthly_exports/` : `sales_2025_01.csv` to `sales_2025_12.csv` .
 - `month_end_pack_2025_messy.xlsx` : the workbook from the story, with the four planted errors.
 - `riverstone_sales.pq` : the Power Query M code from section 11.7.
 - `riverstone_measures.dax` : the DAX measures from section 11.8.
 - `pq_practice/sales_export_with_problems.csv` : a broken export for section 11.7.
 - `challenge/riverstone_stockroom_case.xlsx` and `riverstone_stockroom_solution.xlsx` : the guided championship case from section 11.14 and a helper-column solution.
 - `challenge/riverstone_rewards_case.xlsx` and `riverstone_rewards_solution.xlsx` : the timed practice case and its classic-formula solution.
 - `build_ch11_files.py` : the script that builds all of the above from Riverstone's data (seed 20251).
-

The project: rebuild a messy monthly workbook so next month is one refresh

Goal: replace a copy-paste monthly workbook with one that loads its data, applies its rules once, checks itself, and updates with **Refresh All**.

Option A: your own workbook. Pick a report you (or your team) rebuild every month. Make a copy, remove confidential data or replace names with codes, and follow the steps.

Option B: Riverstone's workbook. Use `month_end_pack_2025_messy.xlsx` and `monthly_exports/` .

Steps

1. **Audit first.** Run the section 11.12 checklist on the old workbook. Write down every problem you find, with its effect in rupees. (Riverstone: four problems; the story lists them.)
2. **Load the data with a query.** Excel: **Get Data → From Folder**, combine the twelve files, delete the automatic type step, set types (dates **Using Locale → English (India)**, codes as Text), filter out cancelled orders, add `net_revenue` , and load to a table named `RiverstoneSales` . Google Sheets version: import each CSV into a tab, stack them with `VSTACK` on a `Data` tab, and add `net_revenue` with one `ARRAYFORMULA` .

3. **Add the lookups once.** Merge Customers (segment) and Products (category, unit cost) in Power Query, or use `XL00KUP` in Sheets.
4. **Build the summary as a pivot table** on the query's table: `month_start` in Rows, `net_revenue` in Values, `segment` as a slicer. Add targets with a lookup beside the pivot, or bring the unpivoted **TargetsWide** query into the model. Sheets: a pivot table, or one `QUERY` with `group by K`.
5. **Add measures** (Excel for Windows): `Valid Net Revenue`, `Gross Margin %`, and `Revenue YTD`, with the Calendar table and relationships from section 11.8.
6. **Add checks.** Grand total minus `SUM` of the table (must be 0). Rows loaded against the total lines in the files (330, then 326 after the filter). A check that shows "OK" or "CHECK" in large text at the top of the report sheet.
7. **Write the README.** Purpose, owner, source folder, the rules, how to refresh, and the date of the last change.
8. **Test the refresh.** Remove the December file, refresh, and confirm the total becomes ₹3,895,647.50; put it back, refresh, and confirm ₹4,335,471. Then add a fake thirteenth file with a wrong header and see what your checks say.
9. **Compare old and new.** A short table: old total, new total, and each difference explained.

Your rebuilt report should show 2025 net revenue of **₹4,335,471, 102.3%** of the ₹4,240,000 target, gross margin **₹1,137,221 (26.2%)**, and Wholesale as the largest segment by revenue (39.3%) but the lowest by margin (19.0%).

Stretch goals

- Add a `Source.Name` column to the query and a pivot that counts lines per file, so a missing or half-exported month is visible at a glance.
- Replace the hard-coded folder path with a parameter stored in a cell on the README sheet.
- Build the Sheets version end to end and make its totals match the Excel version.
- Add a rebate calculation with an approximate-match lookup and a note on customers within ₹5,000 of the next band.

You've got it when...

- ☐ You can write `SUMIFS`, `COUNTIFS`, `AVERAGEIFS`, `MAXIFS`, and `MINIFS` with dates, wildcards, blanks, cell references, and OR logic, and switch to `SUMPRODUCT` when a condition needs a calculation.
- ☐ You know which average a question needs (per line, per order, per customer, weighted) and can calculate each.
- ☐ You can write `XL00KUP` for an exact match, a band (approximate match), the last match, several return columns, and a two-way lookup, and explain the same lookups with `INDEX / MATCH`.
- ☐ You can say why `VL00KUP` breaks and when approximate matching is correct.
- ☐ You can build a pivot table in both apps in under two minutes, with a filter, a percentage of total, grouped dates, and a slicer, and pull its numbers into a report with `GETPIVOTDATA`.
- ☐ You know the difference between counting lines and distinct counting, and how to do each.
- ☐ You can write `FILTER`, `UNIQUE`, `SORT`, and `LET` formulas and fix a `#SPILL!` error.

- ☐ You can combine a folder of files in Power Query, set types deliberately, filter, add a column, merge, and refresh.
- ☐ You can explain the difference between a calculated column and a measure, and write `SUMX` , `CALCULATE` , `DIVIDE` , and `TOTALYTD` measures.
- ☐ You can use Goal Seek and build a two-input what-if grid.
- ☐ You can write a `QUERY` formula with `where` , `group by` , `order by` , and `pivot` , and connect files with `IMPORTRANGE` .
- ☐ You can audit an inherited workbook with a checklist and find typed numbers, short ranges, duplicates, and inconsistent rules.
- ☐ You can name the signals that a job should move to SQL, Python, or a BI tool.
- ☐ You can solve the Stockroom case in 30 minutes: parse text codes, build a daily model with input cells, and search settings with a data table.

Recap

- **Keep data long**, separate data from calculations and outputs, and **state each rule once**. Copy-paste workbooks fail because they repeat rules and ranges by hand.
- The **conditional family** (`SUMIFS` , `COUNTIFS` , `AVERAGEIFS` , `MAXIFS` , `MINIFS`) answers most business questions; criteria are text, joined to cells with `&` . `SUMPRODUCT` handles what they can't: OR across columns, calculated conditions, weighted averages, and distinct counts.
- The **everyday toolkit** (`IFS` , `SWITCH` , `TEXTBEFORE` , `EDATE` , `WORKDAY.INTL` , `ISOWEEKNUM` , `RANK.EQ` , rounding) turns raw columns into the fields reports need.
- **XLOOKUP** covers exact matches, bands (`match_mode -1`), last matches (`search_mode -1`), several columns, and nested two-way lookups. **INDEX / MATCH** does the same in every version. **VLOOKUP** counts columns and defaults to approximate: recognize it, then replace it.
- **Pivot tables** map to SQL: Filters → `WHERE` , Rows and Columns → `GROUP BY` , Values → aggregates. Use distinct counts for orders, **Show Values As** for shares and differences, slicers for filtering, and **refresh** Excel pivots.
- **Dynamic arrays** (`FILTER` , `UNIQUE` , `SORT` , `SEQUENCE` , `LET`) return whole tables from one formula; results **spill**, and a blocked spill shows `#SPILL!` .
- **Power Query** records cleanup as applied steps: combine a folder, set types yourself (delete the automatic guess), filter, add columns, merge, unpivot, and **refresh**.
- **Power Pivot** holds related tables in a **data model**. **Measures** calculate in each cell's **filter context**; `CALCULATE` changes that context. Riverstone's gross margin is 26.2% overall but only 19.0% in Wholesale.
- **Goal Seek** works backward to a target; **data tables** show many what-if answers at once; mixed-reference grids do the same in Sheets.
- Google Sheets' `QUERY` , `ARRAYFORMULA` , `IMPORTRANGE` , and **Connected Sheets** cover much of what Power Query and Power Pivot do in Excel.
- **Competitive Excel** rewards the same habits as good analysis, under a clock: read everything first, parse once, keep settings in input cells, build one row per step, check small cases, and search with data tables.
- **Audit** by reconciling, reading formulas, finding constants, checking ranges, and looking for duplicates. **Leave the spreadsheet** when size, sources, users, scheduling, or risk demand it.

Practice exercises

Use `ch11_practice.xlsx` (with the Sales data converted to a table named `Sales`) unless an exercise says otherwise.

Warm-up

1. Match each pivot table area to the SQL clause it corresponds to: Filters, Rows, Columns, Values. Which area has no single SQL clause?
2. `=VLOOKUP(C2, Customers!A2:D25, 4, FALSE)` returns each customer's segment. Someone inserts a new column between `customer_name` and `city` on the Customers sheet. What does the formula return now, and what formula would have kept working?
3. What does `F2#` refer to, and what causes a `#SPILL!` error?
4. For each, say whether it should be a calculated column or a measure: (a) a customer's rebate band, (b) gross margin %, (c) revenue year to date, (d) each line's product cost.
5. Name two things Excel can do that Google Sheets can't, and the Sheets alternative for each; then one thing the other way round.

Core

6. With one `XLOOKUP`, return the name, city, and segment for customer `0014`.
7. Find Sharma Hardware's (`0001`) first and last order dates with `XLOOKUP`. What does the same lookup return for Home Plus (`0016`), and how would you make it readable?
8. Using the RebateBands sheet and an approximate match, find the band for each customer's 2025 net revenue (excluding cancelled orders). How many customers are in each band, what's the total rebate, and how far is Western Logistics from the next band?
9. Build a pivot table of net revenue by segment (rows) and quarter (columns), excluding cancelled orders. What was Wholesale's Q4 revenue, and what share of the year's revenue did Wholesale contribute?
10. In a pivot, count order lines by segment, then count distinct orders by segment. Give both sets of numbers and explain the difference.
11. Build a pivot of net revenue by sales rep and quarter. Which rep had the highest Q4, and how much revenue had no rep?
12. Use a pivot to find the top three products by net revenue, and the product that sold the least.
13. Write a `FILTER` formula that returns November's Wholesale order lines. How many rows does it return, and what's their total net revenue? Write the Google Sheets version too.
14. Write a formula that lists every customer who ordered in 2025, sorted A to Z. How many names does it return, and why not 24?
15. In Google Sheets, write a `QUERY` that returns net revenue by segment, excluding cancelled orders, largest first. Then change it to spread quarters across the top.
16. Build the Power Query from section 11.7 on `monthly_exports/`. How many rows does it load before and after the cancelled filter, and what's the total?
17. Refresh test: remove `sales_2025_12.csv` from the folder and refresh. What are the new total and the new Q4 total? What should you check after every real refresh?
18. Unpivot the TargetsWide sheet in Power Query. How many rows result, and what do they total?

19. In Power Pivot, create `Valid Net Revenue`, `Gross Margin`, and `Gross Margin %`. What's the overall gross margin %, and which segment and which category have the lowest?
20. Create `Revenue YTD` with a date table. What does it show at the end of June and the end of September?
21. Use Goal Seek to find how many extra Industrial Crates (₹1,400, 8% discount) June needed to reach its ₹300,000 target. Check the answer.
22. Build the price-by-volume what-if grid from section 11.9. What's the value for a 3% price rise with 5% more volume? Which combination in the grid falls below 2025's revenue even though prices rose?

Stretch

23. Audit `month_end_pack_2025_messy.xlsx` without reading the story. List every error you find, its effect in rupees, and the corrected total and percentage of target. Which error has no effect on 2025's numbers, and why does it still matter?
24. Build a month-by-segment summary grid (months down, segments across) and write a two-way lookup, with `INDEX / MATCH` and with nested `XL00KUP`, that returns Wholesale's October revenue.
25. Write a `GROUPBY` formula (Microsoft 365) that returns revenue by segment excluding cancelled orders. What does Retail show if you leave out the filter, and why?

Think about it (no spreadsheet needed)

26. The finance team emails a 40-tab workbook every month: one tab per branch, each pasted from a different system, with a Summary tab. Which three changes from this chapter would you make first, and which signals from section 11.13 suggest it should eventually leave Excel?
27. Western Logistics is ₹1,272.50 below the Silver rebate band. The sales rep asks you to "count their cancelled order so they qualify". What do you say, and what information would help the business decide?
28. A manager says the pivot table in last week's report shows a different Q3 total from the one on your screen today, although "nobody changed anything". List three possible explanations and how you'd check each.

Formula drills (sections 11.2–11.7)

Work these on the Sales table. Write each answer as one formula first, then check it another way (a pivot, a filter, or the status bar).

29. Net revenue from Retail **or** Hospitality customers, excluding cancelled orders. Write it two ways: two `SUMIFS` added together, and one `SUMIFS` with an array constant.
30. Net revenue (excluding cancelled orders) from every product whose name contains "Box", and how many lines that is.
31. For Wholesale and Hospitality, calculate the average net revenue **per line** and the average **per order**. Why are they so different, and which would you quote to a sales head?
32. Calculate gross sales before discount, the simple average discount %, and the revenue-weighted average discount %. Which is the "real" average discount?
33. Count the **distinct customers** who ordered in each segment with `SUMPRODUCT` and `COUNTIFS`.
34. Rank Metro Mart among customers by 2025 net revenue with `RANK.EQ` (or `COUNTIFS`).
35. Find Harbour Traders' first and last order dates with `MINIFS` and `MAXIFS`.
36. As of 31 December 2025, put each customer into a recency bucket (0–30, 31–60, 61–90, over 90 days since their last order). How many customers are in each bucket, and who has gone quiet for more than 90 days?

37. With a two-condition lookup, return the net revenue of the Industrial Crate line (product 105) on order 10107.
38. Build a segment-by-quarter pivot and use `GETPIVOTDATA` to return Farah Khan's Q3 revenue from a rep-by-quarter pivot. What happens to your formula if someone removes `quarter` from the pivot?
39. In `month_end_pack_2025_messy.xlsx`, replace the Summary's twelve different formulas with one `SUMIFS(INDIRECT(...))` formula filled down. Which of the four planted errors does it fix, and which remains?
40. In Power Query, group the valid sales by `month_start` and `segment` with net revenue, line count, and distinct order count. How many rows result, and what does October's Retail row show?
41. What do `WEEKNUM` (type 1) and `ISOWEEKNUM` return for 29 December 2025, and why does it matter for a weekly report?
42. Create the `NETREV` `LAMBDA` from section 11.6 and use it to calculate order 10003's second line (45 units at ₹750, 5% discount).

Timed case (section 11.14)

43. Solve **Riverstone Rewards** (`challenge/riverstone_rewards_case.xlsx`) in 45 minutes: answer L1–L8 and the four bonus questions.

Key terms

lookup array · return array · match mode · search mode · exact match · approximate match · band (tier) table · `INDEX` · `MATCH` · two-way lookup · `VLOOKUP` · `HLOOKUP` · long data · wide data · pivot table · Rows area · Columns area · Values area · Filters area · distinct count · Show Values As · date grouping · value filter · slicer · timeline · `GETPIVOTDATA` · pivot chart · refresh · dynamic array · spill range · `#SPILL!` · spill reference (`#`) · `FILTER` · `UNIQUE` · `SORT` · `SORTBY` · `SEQUENCE` · `LET` · `GROUPBY` · `PIVOTBY` · Power Query · query · applied steps · combine files · data type detection · locale · merge queries · join kind · append queries · unpivot · parameter · M language · load to · Power Pivot · data model · relationship · one-to-many · date table · DAX · calculated column · measure · `SUMX` · `CALCULATE` · `DIVIDE` · `RELATED` · filter context · time intelligence · `TOTALYTD` · what-if analysis · Goal Seek · Solver · data table · Scenario Manager · `QUERY` · Google Visualization API Query Language · `ARRAYFORMULA` · `IMPORTRANGE` · `VSTACK` · Connected Sheets · data warehouse · ODBC · README sheet · inconsistent formula · check cell · Go To Special · Inquire · audit

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 12, Databases & SQL Foundations:** the pivot table's `GROUP BY`, the lookup's `JOIN`, and `QUERY`'s clauses in a real database, for data too big or too shared for a spreadsheet.
- **Chapter 13, SQL for Real Analysis:** running totals, rankings, and month-over-month changes with window functions, the SQL versions of **Show Values As** and `TOTALYTD`.
- **Chapter 14, Data Cleaning & Preparation:** the messy data this chapter assumed away (spelling variants, mixed dates, duplicates), in Power Query, SQL, and pandas.

- **Chapter 16, Business Intelligence with Power BI:** the same Power Query and DAX engines, with a star schema, `CALCULATE` in depth, and dashboards that refresh on a schedule.
- **Chapter 18, Python for Analysts:** `pandas.merge`, `groupby`, and `pivot_table`, which do in code what lookups and pivot tables do here.
- **Chapter 19, Spreadsheet Automation:** macros and scripts for the steps Power Query can't do, such as saving a PDF and emailing it.
- **Chapter 20, Automating Reports & Delivering Insights:** scheduling the refresh this chapter still starts by hand.
- **Interview preparation:** the Excel, Google Sheets, VBA & BI Question Bank (Chapter 70) includes live pivot table, `XLOOKUP`, `INDEX / MATCH`, Power Query, and DAX tasks, and "how would you automate this monthly report?"

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Filters → `WHERE` (rows removed before summarizing). Rows → `GROUP BY` (one row per value). Values → the aggregate (`SUM`, `COUNT`, `AVG`). **Columns** has no single SQL clause: it's also a `GROUP BY` field, but its values are spread across the top as separate columns, which in SQL takes `CASE` expressions (Chapter 12) or a `pivot` in Sheets' `QUERY`.
2. The table `A2:D25` expands to `A2:E25`, but the column number stays 4, which now points at `city` instead of `segment`. The formula returns cities, with no error. `=XLOOKUP(C2, Customers!A2:A25, Customers!E2:E25)` or `=INDEX(Customers!E2:E25, MATCH(C2, Customers!A2:A25, 0))` would have kept working, because Excel updates the column reference (`D` becomes `E`) when the column moves.
3. `F2#` refers to the entire spill range of the dynamic array formula in `F2`, whatever size it currently is. `#SPILL!` means the result can't spill because at least one cell in the space it needs isn't empty (a typed value, another formula, or merged cells), or because the formula is inside an Excel table.
4. (a) **Calculated column** (or a column in Power Query): it's an attribute you'd filter or group by. (b) **Measure**: a ratio must be recalculated for each pivot cell, not added up. (c) **Measure**: it depends on the dates in the filter context. (d) **Calculated column** is reasonable (`quantity × RELATED(unit_cost)`), though a measure with `SUMX` avoids storing it.
5. Excel-only examples: Power Query (Sheets alternative: `IMPORTRANGE`, `VSTACK`, and `QUERY`, or Apps Script, or cleaning in SQL/Python first); Power Pivot and DAX (Sheets alternative: `QUERY` on pre-joined data, Connected Sheets, Looker Studio); Data Table (a mixed-reference grid); `GROUPBY` (`QUERY` with `group by`). Sheets-only examples: `QUERY`, `IMPORTRANGE`, `IMPORTHTML / GOOGLEFINANCE`, and Google Forms.
6. `=XLOOKUP("0014", Customers!A2:A25, Customers!B2:D25)` returns three cells: **Deccan Packaging · Hyderabad · Wholesale**. In Excel the result spills across; in Sheets it fills three cells.
7. First: `=XLOOKUP("0001", Sales[customer_code], Sales[order_date])` → **2025-01-24** (order 10007). Last: `=XLOOKUP("0001", Sales[customer_code], Sales[order_date], "never", 0, -1)` → **2025-12-05** (order 10161). This relies on the table being sorted by date; `MAXIFS(Sales[order_date], Sales[customer_code], "0001")` gives the latest date regardless of order. Home Plus returns `#N/A` without the fourth argument; with `"never"` (or `"no orders in 2025"`) it's readable.

8. First calculate each customer's revenue (a pivot, or `SUMIFS` next to `SORT(UNIQUE(...))`), then `=XLOOKUP(G2, RebateBands!A2:A4, RebateBands!C2:C4, , -1)` for the band and `...B2:B4...` for the rate. **Gold 2** (Sharma Hardware ₹502,775.00 and Harbour Traders ₹412,680.50), **Silver 6** (Northgate Distributors, Green Leaf Hotels, Metro Mart, Coastal Foods, Deccan Packaging, Fresh Bowl Kitchens), **Standard 15**. Total rebate: **₹37,642.03** (for example Sharma Hardware: $2\% \times ₹502,775 = ₹10,055.50$). Western Logistics has ₹248,727.50, which is **₹1,272.50** short of ₹250,000. The common mistake is an exact-match lookup, which returns `#N/A` for every customer.

9. Wholesale Q4: **₹756,751.00**. Wholesale's share of the year: $₹1,702,658.50 \div ₹4,335,471 = 39.3\%$ (**Show Values As → % of Grand Total** in Excel, **Show as → % of grand total** in Sheets). If your grand total is ₹4,398,121, the cancelled filter is missing.

10. Count of `order_id` (lines): Hospitality **107**, Retail **114**, Wholesale **105**, total 326. Distinct count (orders): Hospitality **59**, Retail **59**, Wholesale **55**, total **173**. An order with several products has several lines, so counting lines overstates orders. Excel needs the Data Model for **Distinct Count**; Sheets uses **COUNTUNIQUE**.

11. Q4 by rep: Rahul Mehta **₹682,889.00**, Farah Khan ₹607,959.50, Neha Kulkarni ₹367,820.00. **Rahul Mehta** had the highest Q4. Revenue with no rep: **₹204,502.50** for the year (it appears as *(blank)* in Excel's pivot). Rahul's Q4 was more than a third of his annual ₹1,494,000.50.

12. Top three: **Storage Box 25L ₹908,212.50**, **Storage Box 10L ₹860,946.00**, **Industrial Crate ₹795,830.00**. The least: **Garden Chair ₹33,637.50**.

13. Excel: `=FILTER(Sales[[order_id]:[net_revenue]], (Sales[month_start]=DATE(2025,11,1))* (Sales[segment]="Wholesale"), "no rows")`. Sheets: `=FILTER(Sales!A2:J331, Sales!K2:K331=DATE(2025,11,1), Sales!N2:N331="Wholesale")`. **15 rows**, totaling **₹343,685.50** (check it with `=SUM(FILTER(Sales[net_revenue], (Sales[month_start]=DATE(2025,11,1))*(Sales[segment]="Wholesale")))`). No November Wholesale lines were cancelled, so no status condition is needed here, but adding `*(Sales[status]<>"Cancelled")` is the safer habit.

14. `=SORT(UNIQUE(Sales[customer_name]))` returns **23** names, starting Blue Bay Cafe, City Needs Store, Coastal Foods. Riverstone has 24 customers, but **Home Plus** placed no orders in 2025, so it doesn't appear in the sales data. To list all customers, use the Customers sheet; to find customers with no orders, compare the two lists (Chapter 12's anti-join).

15. `=QUERY(Sales!A1:S331, "select N, sum(J) where H <> 'Cancelled' group by N order by sum(J) desc label sum(J) 'net_revenue'", 1)` → Wholesale 1702658.5, Retail 1488773.75, Hospitality 1144038.75. Quarters across the top: `"select N, sum(J) where H <> 'Cancelled' group by N pivot L"`, which gives Q1–Q4 columns matching Figure 11.3.

16. **330 rows** before the filter ($15 + 20 + 18 + 21 + 29 + 22 + 29 + 28 + 40 + 38 + 35 + 35$), **326** after, total **₹4,335,471**. If the total is right but `customer_code` shows **2** instead of **0002**, the automatic **Changed Type** step is still there.

17. Without December: total **₹3,895,647.50** ($₹4,335,471 - ₹439,823.50$) and Q4 **₹1,314,478.75** (after refreshing the pivot too). After every real refresh, check the rows loaded against the lines in the files, lines per file (a missing or partial month), the grand total against an independent number, and that no step shows an error.

18. Select `year` → **Unpivot Other Columns: 12 rows** (Jan to Dec), totaling **₹4,240,000**. Rename `Attribute` and `Value` to `month` and `target`, and convert `month` to a real date if you'll join it to the Calendar table.

19. Overall **26.2%** (₹1,137,221 on ₹4,335,471). Lowest segment: **Wholesale 19.0%** (Hospitality 32.4%, Retail 29.8%). Lowest category: **Industrial 13.6%** (Kitchen 33.2%, Storage 26.9%, Furniture 24.2%). A common error is summing a gross margin % column, which gives meaningless totals; the measure recalculates for each cell.

20. End of June **₹1,460,879.75**; end of September **₹2,581,168.75**; December ₹4,335,471. If YTD equals each month's own revenue, the Calendar table isn't marked as a date table or isn't related to `order_date`.

21. Model: `=B1+B2*B3*(1-B4/100)` with B1 = 186928, B3 = 1400, B4 = 8. Goal Seek sets B5 to 300000 by changing B2 : **₹7.79**, so **88 crates**. Check: $88 \times ₹1,288 = ₹113,344$; $₹186,928 + ₹113,344 = ₹300,272$, which reaches the target, while 87 crates give ₹298,984, which doesn't.

22. Price +3%, volume +5%: **₹4,688,812** ($4,335,471 \times 1.03 \times 1.05$). **Price +5% with volume -5%** gives ₹4,324,632, below 2025's ₹4,335,471, because $1.05 \times 0.95 = 0.9975$. (Price +3% with -5% volume gives ₹4,242,258, also below.) The Sheets version: `=B$1*(1+$B6)*(1+C$5)` copied across the grid.

23. Four errors, as in the story: **October** range stops at row 30 (9 lines, **-₹136,023**); **December** last five lines pasted twice (**+₹46,555**); **April** uses `SUM`, including cancelled order 10034 (**+₹24,800**); **November** typed as 633,480 (**+₹72**). Net effect -₹64,596. Corrected total **₹4,335,471, 102.3%** of target (the messy version: ₹4,270,875, 100.7%). **March** also uses `SUM` but had no cancelled orders, so it's right by luck; it matters because the rule is wrong and will fail the first time a March order is cancelled.

24. With months in A5:A16 and segments in B4:D4 : `=INDEX(B5:D16, MATCH(DATE(2025,10,1), A5:A16, 0), MATCH("Wholesale", B4:D4, 0))` and `=XLOOKUP("Wholesale", B4:D4, XLOOKUP(DATE(2025,10,1), A5:A16, B5:D16))`. Both return **₹207,827**. Build the grid with a pivot pasted as values, `PIVOTBY`, a `QUERY` pivot, or `SUMIFS` with mixed references.

25. `=GROUPBY(Sales[segment], Sales[net_revenue], SUM, , , , Sales[status]<>"Cancelled")` → Hospitality ₹1,144,038.75, Retail ₹1,488,773.75, Wholesale ₹1,702,658.50, total ₹4,335,471. Without the filter, Retail shows **₹1,551,423.75**, because both cancelled orders (10034, ₹24,800, and 10131, ₹37,850) were from Retail customers.

26. First changes: (1) load each branch's data with Power Query (or one query per source system) into one long table with a `branch` column, instead of pasted tabs; (2) apply the rules once and build the Summary as a pivot or measures; (3) add check cells and a README with the refresh steps. Signals that it should leave Excel: several source systems joined every month, many people depending on it, a monthly manual refresh that should be scheduled, and the cost of an error in a finance report. A database or warehouse with SQL, feeding Power BI or a refreshable workbook, is the likely destination.

27. Say no: a cancelled order isn't revenue under the report's rule, and changing the rule for one customer would make the rebate report inconsistent (and, for a real rebate, it would be paid on sales that didn't happen). Offer what would help the decision instead: how close other customers are to each band, the cost of lowering the Silver threshold or adding a grace margin for everyone, and whether Western Logistics' 2026 orders are on track to reach the band. Chapter 24 covers handling "can you just change the number?"

28. Possible explanations: (1) **the pivot was refreshed** after new or corrected data arrived (check the query's last refresh time and the data's row count); (2) **a filter or slicer differs** (for example, cancelled orders included in one version; compare the filter settings and the grand totals); (3) **the source data was changed** (a late order, a corrected discount, a re-exported month; compare the two data snapshots with version history or the file dates). A dated "as of" note and a row-count check cell on the report prevent the argument next time.

29. `=SUMIFS(Sales[net_revenue], Sales[segment], "Retail", Sales[status], "<>Cancelled") + SUMIFS(Sales[net_revenue], Sales[segment], "Hospitality", Sales[status], "<>Cancelled")` and `=SUM(SUMIFS(Sales[net_revenue], Sales[segment], {"Retail","Hospitality"}, Sales[status], "`

<>"Cancelled")) . Both return **₹2,632,812.50** (₹1,488,773.75 + ₹1,144,038.75). The array constant makes SUMIFS return two totals, and SUM adds them. A common mistake is two criteria pairs on the same column, which asks for rows that are Retail and Hospitality at once and returns 0.

30. =SUMIFS(Sales[net_revenue], Sales[product_name], "*Box*", Sales[status], "<>"Cancelled") returns **₹2,315,788.50**, and the matching COUNTIFS returns **168** lines: Storage Box 10L, Storage Box 25L, and Lunch Box Set. Wildcards aren't case-sensitive, so "box" gives the same result.

31. Per line: Wholesale **₹16,215.80**, Hospitality **₹10,691.95** (AVERAGEIFS). Per order: Wholesale **₹30,957.43** (₹1,702,658.50 ÷ 55 orders), Hospitality **₹19,390.49** (₹1,144,038.75 ÷ 59). An order has about two lines on average, so per-order values are roughly double. A sales head thinks in orders (and the business-wide average order value is ₹25,060.53), so quote the per-order figure and say how it was calculated.

32. Gross sales before discount: =SUMPRODUCT(Sales[quantity], Sales[unit_price], --(Sales[status] <>"Cancelled")) → **₹4,548,725**. Simple average discount: =AVERAGEIFS(Sales[discount_pct], Sales[status], "<>"Cancelled") → **4.06%**. Weighted: =1 - 4335471/4548725 → **4.69%**. The weighted figure is the real average: it's the share of list value Riverstone gave away, and larger lines carried larger discounts.

33. =SUMPRODUCT((Sales[segment]="Wholesale") * (Sales[status]<>"Cancelled") / COUNTIFS(Sales[customer_name], Sales[customer_name], Sales[segment], Sales[segment], Sales[status], Sales[status])) for each segment: Hospitality **9**, Retail **8**, Wholesale **6**, total 23. In Microsoft 365, =ROWS(UNIQUE(FILTER(Sales[customer_name], (Sales[segment]="Wholesale")*(Sales[status] <>"Cancelled")))) is easier to read.

34. With customer names in F2:F24 and their revenue in G2:G24: =RANK.EQ(G13, \$G\$2:\$G\$24, 0) for Metro Mart gives **5** (after Sharma Hardware, Harbour Traders, Northgate Distributors, and Green Leaf Hotels). =COUNTIFS(\$G\$2:\$G\$24, ">"&G13)+1 gives the same rank without RANK.EQ .

35. =MINIFS(Sales[order_date], Sales[customer_name], "Harbour Traders", Sales[status], "<>"Cancelled") → **2025-04-09**; MAXIFS → **2025-12-14**. Format both cells as dates; MINIFS returns a date serial number.

36. Last order per customer with MAXIFS , then =DATE(2025,12,31)-last_order for days, then =IFS(days<=30,"0-30",days<=60,"31-60",days<=90,"61-90",TRUE,"90+") . Counts: **0-30: 15, 31-60: 4, 61-90: 1, 90+: 3** (23 customers). Over 90 days: **City Needs Store, Om Sai Provisions, and Sunrise Caterers**. Sharma Hardware, for comparison, last ordered 26 days before year-end.

37. =XLOOKUP(1, (Sales[order_id]=10107) * (Sales[product_id]=105), Sales[net_revenue]) → **₹56,700**. In any version: =INDEX(Sales[net_revenue], MATCH(1, INDEX((Sales[order_id]=10107)*(Sales[product_id]=105), 0), 0)) . Because the pair is unique, =SUMIFS(Sales[net_revenue], Sales[order_id], 10107, Sales[product_id], 105) also works, and it's the fastest to type in a competition.

38. =GETPIVOTDATA("net_revenue", \$A\$3, "sales_rep", "Farah Khan", "quarter", "Q3") → **₹492,618.75**. If quarter is removed from the pivot (or a slicer hides Farah Khan), the item is no longer visible and the formula returns **#REF!** . Keep report-feeding pivots on a locked sheet and check GETPIVOTDATA cells with ISERROR in a check cell.

39. With month names in A4:A15: =SUMIFS(INDIRECT("'"&A4&"'!J2:J100"), INDIRECT("'"&A4&"'!H2:H100"), "<>"Cancelled") , filled down. It fixes three errors: **October** (the range now reaches row 100: ₹681,070.75), **April** (the same rule everywhere: ₹210,281.50), and **November** (no typed number: ₹633,408). It doesn't fix **December's** double paste, because the duplicated lines are real rows on the sheet: December still shows ₹486,378.50 instead of ₹439,823.50. A formula can only be as right as its data; the duplicate needs a check such as COUNTIFS(order_id, A2, product_id, D2)>1 .

40. Home → Group By → Advanced, grouping by `month_start` and `segment`, with **Sum** of `net_revenue`, **Count Rows**, and a distinct count of `order_id` (see the tool note in section 11.7: a second Group By, or `each List.Count(List.Distinct([order_id]))` in the Advanced Editor). **36 rows** (12 months × 3 segments). October Retail: **₹283,025**, 12 lines, 7 orders.

41. `WEEKNUM(DATE(2025,12,29), 1)` returns **53** (weeks start on Sunday, and week 1 contains 1 January). `ISOWEEKNUM` returns **1**: under the ISO standard, 29 December 2025 belongs to week 1 of 2026. A weekly report that mixes the two splits or merges a week at the year boundary, so agree one system and write it on the report.

42. Formulas → Name Manager → New, name `NETREV`, refers to `=LAMBDA(q, p, d, q * p * (1 - d/100))`. `=NETREV(45, 750, 5)` returns **32062.5**, matching Chapter 10's hand check. `LAMBDA` is in Microsoft 365 and Google Sheets (via **Data → Named functions**).

43. Riverstone Rewards answers:

Level	Answer
L1	173 orders; 70 worth ₹25,000 or more
L2	43,287 base points; Sharma Hardware (5,019)
L3	24 Gold-tier orders; 56 Silver (93 Standard)
L4	50,403 tier points
L5	129 streak orders; 76,203 points
L6	79,703 order points
L7	3 customers (City Needs Store, Om Sai Provisions, Sunrise Caterers); 2,273 points expire
L8	Sharma Hardware, 9,794 ; 9 customers with 4,000 or more
B1	Sharma Hardware , on order 10088 (12 August 2025, running total 5,346)
B2	October 2025, 12,446 points
B3	3 customers (Coastal Foods, Metro Mart, Sharma Hardware)
B4	4 (Metro Mart, 7,062 points)

The most common wrong answers come from four traps: counting cancelled order 10131 (it's in the Lines sheet); using a customer's *total* revenue for the tier instead of revenue from earlier orders only (Sharma Hardware's first order is Standard, not Gold); giving the streak bonus to January orders (there's no December 2024 in the data); and expiring points for customers who ordered in Q4. Check L3 by hand for one customer: Sharma Hardware moves to Silver on order 10018 (₹112,750 of earlier orders) and to Gold on order 10088 (₹260,372.50).